

PHASED_REMEDIATION_PLAN

- [Helix Cluster OS — Phased Remediation Plan](#)
 - [Table of Contents](#)
- [Executive Summary](#)
 - [Scope and Purpose](#)
 - [Current State](#)
 - [Phase Overview](#)
 - [Critical Path](#)
- [Remediation Principles](#)
 - [P1: No Regression on Foundation](#)
 - [P2: Sink-Side Evidence Required](#)
 - [P3: Bottom-Up Wiring](#)
 - [P4: Constitution Compliance First](#)
 - [P5: Autonomous vs. Owner-Gated](#)
- [Phase 0: Stabilize Foundation](#)
 - [Objective](#)
 - [Entry Criteria](#)
 - [Exit Criteria](#)
 - [PH0-001: SQL Schema Drift Guard \(HXC-1639\)](#)
 - [PH0-002: SWIM Stop/Leave Double-Close \(F1\)](#)
 - [PH0-003: SWIM HealthyMembers Data Race \(F2\)](#)
 - [PH0-004: SWIM Untracked Goroutine \(F3\)](#)
 - [PH0-005: SWIM Lock-Order Fragility \(F4\)](#)
 - [PH0-006: PTY Double-Close \(F5\)](#)
 - [PH0-007: Scheduler Unbounded Map \(F6\)](#)
 - [PH0-008: EventBus Unbounded Goroutines \(F7\)](#)
 - [PH0-009: Race Gate Full](#)
 - [PH0-010: Whole-Tree Build/Vet Gate](#)
- [Phase 1: Dead-Code Resolution](#)
 - [Objective](#)
 - [Entry Criteria](#)
 - [Exit Criteria](#)
 - [Wiring Tiers](#)
 - [Tier 1: Control-Plane Spine \(HIGHEST PRIORITY — Wire-In\)](#)
 - [Tier 2: GPU Stack \(HIGH PRIORITY — Wire-In\)](#)
 - [Tier 3: Scheduler Helpers \(HIGH PRIORITY — Wire-In\)](#)
 - [Tier 4: Federation & Data-Plane \(MEDIUM PRIORITY — Wire-In\)](#)
 - [Tier 5: Security Extensions \(HIGH PRIORITY — Wire-In\)](#)
 - [Tier 6: Marketplace & Economics \(MEDIUM PRIORITY — Wire-In/Document\)](#)
 - [Tier 7: Infrastructure & Operations \(MEDIUM PRIORITY — Wire-In\)](#)
 - [Tier 8: Caching & Performance \(MEDIUM PRIORITY — Wire-In\)](#)
 - [Tier 9: LLM & Inference \(MEDIUM PRIORITY — Wire-In\)](#)
 - [Tier 10: Edge Computing \(MEDIUM PRIORITY — Wire-In\)](#)

- [Tier 11: Testing Infrastructure \(MEDIUM PRIORITY — Wire-In\)](#)
 - [Tier 12: Utility & Specialized \(LOW PRIORITY — Document-As-Library or Prune\)](#)
 - [Triage Execution Procedure](#)
 - [Pruning Decision Matrix](#)
- [Phase 2: Test Coverage to Practical Maximum](#)
 - [Objective](#)
 - [Entry Criteria](#)
 - [Exit Criteria](#)
 - [PH2-001: cmd/helix-gate/main_test.go](#)
 - [PH2-002: pkg/crypto/fuzz_test.go](#)
 - [PH2-003: pkg/security/fuzz_test.go](#)
 - [PH2-004: pkg/hybridkex/fuzz_test.go](#)
 - [PH2-005: Integration test suite for pkg/etcd](#)
 - [PH2-006: Integration test for pkg/lock/EtcdLocker](#)
 - [PH2-007: E2E test for workload placement](#)
 - [PH2-008: E2E test for session lifecycle](#)
 - [PH2-009: Chaos test for SWIM partition and recovery](#)
 - [PH2-010: Chaos test for etcd leader election failover](#)
 - [PH2-011 through PH2-050: Additional Test Files](#)
 - [PH2-051: Frontend Tests](#)
 - [PH2-052: Stress Test Suite \(48-hour sustained load\)](#)
 - [PH2-053: Soak Test Plan \(7-day GPU fleet\)](#)
 - [PH2-054: Mutation Test Expansion](#)
- [Phase 3: Challenges Integration](#)
 - [Objective](#)
 - [PH3-001: Add challenges submodule to go.work](#)
 - [PH3-002: Create banks/helix_cluster/ YAML definitions \(30+ challenges\)](#)
 - [PH3-003: Wire challenge runner into Makefile](#)
 - [PH3-004: Create shell script challenges for infrastructure](#)
 - [PH3-005: Create userflow challenges for API/gRPC](#)
 - [PH3-006: Anti-bluff scanner integration](#)
 - [PH3-007: Mutation testing integration](#)
 - [PH3-008: Evidence collection pipeline](#)
- [Phase 4: Responsiveness](#)
 - [Objective](#)
 - [PH4-001: F8 fix — EventBus Subscribe buffered delivery](#)
 - [PH4-002: F9 fix — TieredCache hot tier size cap](#)
 - [PH4-003: F10 fix — Per-event timer allocation optimization](#)
 - [PH4-004: Tree-wide lazy-init audit](#)
 - [PH4-005: Tree-wide bounded pool audit](#)
- [Phase 5: Security Scanning Deep](#)
 - [Objective](#)
 - [PH5-001: Promote gosec to make security-scan](#)
 - [PH5-002: Wire govulncheck into CI](#)
 - [PH5-003: Trivy scan integration](#)
 - [PH5-004: SonarQube setup \(when tokens available\)](#)
- [Phase 6: Bucket A Registry Backlog](#)
 - [Objective](#)
 - [Bucket A Item Categories](#)

- [Detailed Bucket A Item Registry](#)
 - [CRITICAL Fixes \(3 items\)](#)
 - [Stub Bluff Replacements \(8 items\)](#)
 - [Wiring Completions \(15 items\)](#)
 - [Test Additions \(20 items\)](#)
 - [Security Fixes \(10 items\)](#)
 - [Documentation Updates \(10 items\)](#)
 - [Performance Fixes \(5 items\)](#)
 - [Cross-Platform Fixes \(7 items\)](#)
 - [CRIT-1: Replace JWT Stub Tokens with Real Verification](#)
 - [CRIT-2: Add gRPC Health Service to cmd/helix-health](#)
 - [CRIT-3: Fix Build Service Import Paths](#)
- [Phase 7: Documentation Continuous](#)
 - [Objective](#)
 - [PH7-001: docs_chain engine integration](#)
 - [PH7-002: Export format pipeline \(MD→PDF→HTML→DOCX\)](#)
 - [PH7-003: Diagram generation automation](#)
- [Phase 8: Bucket B Infrastructure](#)
 - [Objective](#)
 - [Resource Requirements](#)
 - [Sub-Phases](#)
 - [Phase 8A: Kubernetes Deployment \(4-6 weeks\)](#)
 - [Phase 8B: GPU Hardware Testing \(4-6 weeks\)](#)
 - [Phase 8C: Cross-Platform Testing \(2-4 weeks\)](#)
 - [Phase 8D: Cloud Deployment \(2-4 weeks\)](#)
- [Phase 9: Bucket C Governance](#)
 - [Objective](#)
 - [PH9-001: CI re-enable decision \(HXC-105/HXC-1262\)](#)
 - [PH9-002: PRR ≥95% closure](#)
 - [PH9-003: Release authorization](#)
- [Cross-Phase Dependencies](#)
 - [Dependency Graph](#)
 - [Parallel Execution Opportunities](#)
- [Risk Register](#)
- [Resource Estimation](#)
 - [Person-Hours by Phase](#)
 - [Timeline](#)
 - [Optimistic \(all owner decisions immediate, all resources available\)](#)
 - [Realistic \(owner decisions take 1-2 weeks, some resources delayed\)](#)
 - [Pessimistic \(owner decisions take 4+ weeks, resources severely delayed\)](#)
- [Appendix A: Item Registry Cross-Reference](#)
- [Appendix B: Constitution Compliance Map](#)
- [Appendix C: Glossary](#)

Helix Cluster OS — Phased Remediation Plan

Field	Value
Document ID	PRP-001
Revision	1.0
Date	2026-03-05
Classification	INVESTIGATION — INTERNAL
Authors	Autonomous Remediation Planning Agent
Status	FINAL
Codebase Version	v0.1.0-dev
Go Version	1.25.0 / toolchain go1.26.4
Repository	github.com/HelixDevelopment/helix_cluster
Constitution Reference	§1.1, §7.1, §11.4, CLAUDE-1/2/3, CONST-035, CONST-050

Table of Contents

1. [Executive Summary](#)
2. [Remediation Principles](#)
3. [Phase 0: Stabilize Foundation \(2–3 weeks\)](#)
4. [Phase 1: Dead-Code Resolution \(4–6 weeks, OWNER-GATED\)](#)
5. [Phase 2: Test Coverage to Practical Maximum \(6–8 weeks\)](#)
6. [Phase 3: Challenges Integration \(4–6 weeks\)](#)
7. [Phase 4: Responsiveness \(2–3 weeks\)](#)
8. [Phase 5: Security Scanning Deep \(2–3 weeks\)](#)
9. [Phase 6: Bucket A Registry Backlog \(8–12 weeks\)](#)
10. [Phase 7: Documentation Continuous \(ongoing\)](#)
11. [Phase 8: Bucket B Infrastructure \(12–16 weeks, OWNER-PROVISIONED\)](#)
12. [Phase 9: Bucket C Governance \(2–4 weeks, OWNER-DECISION\)](#)
13. [Cross-Phase Dependencies](#)
14. [Risk Register](#)
15. [Resource Estimation](#)
16. [Appendix A: Item Registry Cross-Reference](#)
17. [Appendix B: Constitution Compliance Map](#)

Executive Summary

Scope and Purpose

This document defines the definitive 9-phase remediation plan for the Helix Cluster OS project. It addresses every finding from the Comprehensive Analysis Report (HCAR-001), Consolidated Gap Audit (GAC-001), Test Strategy (TS-100PCT), Constitutional Compliance Report (CCR-001), Challenges Integration Plan (CIP-001), SQL Definitions (SQL-DEFS-001), Architecture Diagrams (ARCH-DIAG-001), and Technical Research Compendium (TRC-001). Each phase is structured with granular work items, specific file paths, code-level specifications, test commands, acceptance criteria, and owner/gate designations.

Current State

Metric	Current	Target	Delta
HXC Registry completed / queued	448 / 240	688 / 0	–240 queued
Production Readiness Review (PRR)	82.5% (66/80)	≥95% (76/80)	+12.5%
Orphaned pkg/ packages	178/212	0/212	–178
Concurrency hazards (confirmed)	10 (F1–F10)	0	–10
CRITICAL service disconnections	3 (CRIT-1/2/3)	0	–3
Anti-bluff pass rate	33% (10/30)	100%	+67%
Mutation test compliance	2% (5/255 pkgs)	100%	+98%
Test type coverage	9/20 types	20/20 types	+11

Metric	Current	Target	Delta
CI workflows active	0/12	12/12	+12
Security scan tools active	1/4 (govulncheck)	4/4	+3
Cross-platform parity	Linux: 85%, macOS: 45%, Win: 5%	100% each	+15/+55/+95
GPU backend coverage	1/4 (NVIDIA partial)	4/4	+3
Constitutional compliance	19.2% (2/26 rules)	100%	+80.8%
Frontend test coverage	0%	≥80%	+80%
Documentation sync	Partial (not enforced)	Continuous (docs_chain)	Full enforcement

Phase Overview

Phase	Name	Duration	Owner	Items	Person-Hours
0	Stabilize Foundation	2-3 weeks	Autonomous	10	~160
1	Dead-Code Resolution	4-6 weeks	OWNER-GATED	30+	~480
2	Test Coverage to Practical Maximum	6-8 weeks	Autonomous	54+	~640
3	Challenges Integration	4-6 weeks	Autonomous	8	~320
4	Responsiveness	2-3 weeks	Autonomous	5	~80
5	Security Scanning Deep	2-3 weeks	Autonomous	4	~60
6		8-12 weeks	Autonomous	78	~960

Phase	Name	Duration	Owner	Items	Person-Hours
	Bucket A Registry Backlog				
7	Documentation Continuous	Ongoing	Autonomous	3	~120
8	Bucket B Infrastructure	12-16 weeks	OWNER-PROVISIONED	152	~1,200
9	Bucket C Governance	2-4 weeks	OWNER-DECISION	10	~40
Total		40-68 weeks		~354	~4,060

Critical Path

The critical path through the remediation plan follows this dependency chain:

Phase 0 (Foundation)

↓

Phase 1 (Dead-Code Resolution) ← OWNER-GATED: requires triage decisions

↓

Phase 2 (Test Coverage) ← depends on Phase 1 for wired packages

↓

Phase 3 (Challenges) ← depends on Phase 2 for test infrastructure

↓

Phase 4 (Responsiveness) ← can parallel with Phase 3

Phase 5 (Security) ← can parallel with Phase 3

↓

Phase 6 (Bucket A) ← depends on Phases 1-5 for infrastructure

↓

Phase 7 (Documentation) ← continuous, integrates with all phases

↓

Phase 8 (Bucket B) ← OWNER-PROVISIONED: requires hardware/cloud

Phase 9 (Bucket C) ← OWNER-DECISION: requires governance rulings

Blocking items on the critical path:

1. **Phase 0 → Phase 1:** All concurrency hazards must be resolved before wiring orphaned code. A data race in an orphaned package that gets wired into a binary becomes a production crash.
 2. **Phase 1 → Phase 2:** Tests cannot be written for packages that haven't been triaged (wire-in vs. prune vs. document). Writing tests for a package that gets pruned is wasted effort.
 3. **Phase 1 owner gate:** The triage of 178 orphaned packages requires owner decisions for each WIRE-IN / PRUNE / DOCUMENT-AS-LIBRARY recommendation. This is the single longest owner-gated step in the plan.
 4. **Phase 9 CI decision:** CI re-enablement (HXC-105/HXC-1262) blocks PRR items 8/9/10/16/60 and is constitutionally gated.
-

Remediation Principles

P1: No Regression on Foundation

Every change must pass the Phase 0 gates before merging: - `go test -race -count=1 ./...` must pass with zero races - `go build ./... && go vet ./...` must pass clean - All existing tests must continue to pass

No phase may introduce new race conditions, build failures, or vet warnings. If a change breaks the foundation, it is reverted immediately.

P2: Sink-Side Evidence Required

Per Constitution §11.4.4 and CLAUDE-1, no work item is considered complete without sink-side evidence. For each item: - The fix must be verified by a test that proves the feature works for end users - A mutation test must verify the test catches breakage - Evidence must be captured in `qa-results/` per the evidence taxonomy

P3: Bottom-Up Wiring

When wiring orphaned packages, dependencies must be resolved before dependents. The “textbook double-orphan” pattern (where package A depends on orphaned package B) requires wiring B before A. The dependency graph must be respected:

```
pkg/etcd → pkg/leader → pkg/lock
pkg/crdt → pkg/deltacrdt → pkg/session
pkg/swim → pkg/splitbrain → pkg/splitbrainalert
```


P4: Constitution Compliance First

Every work item must move the constitutional compliance score upward. Items that would violate CLAUDE-1 (end-user usability), CLAUDE-2 (cross-platform parity), CLAUDE-3 (documentation sync), or §11.4 (anti-bluff) are not acceptable.

P5: Autonomous vs. Owner-Gated

- **Autonomous items** can be executed by subagents without owner approval
- **OWNER-GATED items** require an explicit owner decision before proceeding
- **OWNER-PROVISIONED items** require owner-provisioned resources (hardware, cloud accounts, tokens)
- **OWNER-DECISION items** require a governance ruling

No autonomous agent may bypass an owner gate. When an owner gate is encountered, the agent must stop, document the decision needed, and wait for the owner's ruling.

Phase 0: Stabilize Foundation

Duration: 2–3 weeks

Owner: Autonomous

Prerequisite: None (this is the first phase)

Gate: Zero races, zero build failures, zero vet warnings

Estimated Effort: ~160 person-hours

Objective

Eliminate all concurrency hazards, establish the SQL schema drift guard, and create the zero-regression gate that all subsequent phases must pass. Phase 0 is the foundation upon which every other phase depends. Without it, wiring orphaned code would introduce production crashes, and writing tests would produce flaky results.

Entry Criteria



Codebase builds successfully (`go build ./...`)



Existing tests pass (`go test ./...`)



Makefile build target functional (HXC-1637)

Exit Criteria

- ☐ `go test -race -count=1 ./...` passes with zero data races
 - ☐ `go build ./...` passes clean
 - ☐ `go vet ./...` passes clean
 - ☐ SQL schema drift guard test passes
 - ☐ All 7 confirmed concurrency hazards (F1-F7) resolved
 - ☐ No new test failures introduced
-

PH0-001: SQL Schema Drift Guard (HXC-1639)

Priority: P0 — CRITICAL

HXC Reference: HXC-1639

Constitution Reference: §11.4.106

Estimated Effort: 8 person-hours

Status: Not Started

Problem Statement:

The PostgreSQL schema has two representations that must remain in lockstep: 1. The golang-migrate chain (`001_create_nodes.up.sql` through `015_triggers_and_functions.up.sql`) — the canonical source 2. The consolidated artifact (`0001_primary_schema.sql`) — the in-order concatenation produced by `migrations/postgresql/gen_schema.py`

If these diverge, applying the migration chain produces a different schema than applying the consolidated artifact. This has already happened: the gap audit identified schema drift between the two representations (HXC-1639).

File: `internal/schema/drift_guard_test.go` (NEW)

Implementation Specification:

```
package schema_test

import (
    "os"
    "strings"
    "testing"
```

```

        "github.com/HelixDevelopment/helix_cluster/internal/
        schema"
        "github.com/HelixDevelopment/helix_cluster/migrations/
        postgresql"
    )

    // TestSchemaDriftGuard verifies that the consolidated
    // primary schema
    // and the sequential application of migrations 001-015
    // produce
    // identical CREATE TABLE statements. This test MUST pass
    // before any
    // migration changes are merged.
    //
    // HXC-1639: This test was created because the two
    // representations
    // had drifted, causing ApplyPrimarySchema() to produce a
    // different
    // schema than applying migrations sequentially.
    func TestSchemaDriftGuard(t *testing.T) {
        // Step 1: Load the consolidated primary schema
        primarySchema, err := os.ReadFile("migrations/
        postgresql/0001_primary_schema.sql")
        if err != nil {
            t.Fatalf("failed to read primary schema: %v", err)
        }

        // Step 2: Load and concatenate all migration files in
        // order
        migrationFiles := []string{
            "001_create_nodes.up.sql",
            "002_create_gpu_devices.up.sql",
            "003_create_sessions.up.sql",
            "004_create_session_windows.up.sql",
            "005_create_session_panes.up.sql",
            "006_create_reservations.up.sql",
            "007_create_scheduling_queue.up.sql",
            "008_create_health_snapshots.up.sql",
            "009_create_llm_advisories.up.sql",
            "010_create_audit_log.up.sql",
            "011_create_build_jobs.up.sql",
            "012_create_build_artifacts.up.sql",
            "013_create_users.up.sql",
            "014_create_migration_history.up.sql",
            "015_triggers_and_functions.up.sql",
        }

        var chainSchema strings.Builder

```

```

for _, mf := range migrationFiles {
    content, err := os.ReadFile("migrations/
postgresql/" + mf)
    if err != nil {
        t.Fatalf("failed to read migration %s: %v", mf,
err)
    }
    chainSchema.Write(content)
    chainSchema.WriteString("\n")
}

// Step 3: Parse CREATE TABLE statements from both
sources
primaryTables := parseCreateTables(t,
    string(primarySchema))
chainTables := parseCreateTables(t,
    chainSchema.String())

// Step 4: Assert column parity for each table
for tableName, primaryCols := range primaryTables {
    chainCols, exists := chainTables[tableName]
    if !exists {
        t.Errorf("table %q exists in primary schema but
not in migration chain", tableName)
        continue
    }

    // Compare columns
    if len(primaryCols) != len(chainCols) {
        t.Errorf("table %q: primary schema has %d
columns, migration chain has %d",
            tableName, len(primaryCols), len(chainCols))
    }

    for colName, primaryType := range primaryCols {
        chainType, exists := chainCols[colName]
        if !exists {
            t.Errorf("table %q: column %q exists in
primary schema but not in migration chain",
                tableName, colName)
            continue
        }
        if primaryType != chainType {
            t.Errorf("table %q: column %q type mismatch:
primary=%q, chain=%q",
                tableName, colName, primaryType,
                chainType)
        }
    }
}

```

```

    }
}

// Check for columns in chain but not in primary
for colName := range chainCols {
    if _, exists := primaryCols[colName]; !exists {
        t.Errorf("table %q: column %q exists in
migration chain but not in primary schema",
            tableName, colName)
    }
}
}

// Check for tables in chain but not in primary
for tableName := range chainTables {
    if _, exists := primaryTables[tableName]; !exists {

        t.Errorf("table %q exists in migration chain but not
in primary schema", tableName)
    }
}
}

// parseCreateTables extracts table names and their columns
// with types
// from a SQL schema string. Returns
// map[tableName]map[colName]colType.
func parseCreateTables(t *testing.T, schema string)
    map[string]map[string]string {
    tables := make(map[string]map[string]string)
    // Parse CREATE TABLE statements using a simple state
    // machine
    // that handles parenthesized column definitions.
    //
    // This parser handles:
    // - CREATE TABLE IF NOT EXISTS foo (...)
    // - Column definitions with type: col_name TYPE NOT
    //   NULL DEFAULT ...
    // - CONSTRAINT lines (skipped)
    // - PRIMARY KEY lines (skipped)
    // - INDEX definitions (skipped)
    //
    // For production use, consider using a SQL parser
    // library.
    // This implementation is intentionally simple for the
    // drift guard.

    lines := strings.Split(schema, "\n")

```

```

var currentTable string
var inCreateTable bool

for _, line := range lines {
    trimmed := strings.TrimSpace(line)

    // Detect CREATE TABLE start
    if strings.HasPrefix(trimmed, "CREATE TABLE") {
        inCreateTable = true
        // Extract table name
        parts := strings.Fields(trimmed)
        for i, p := range parts {
            if p == "EXISTS" && i+1 < len(parts) {
                currentTable = strings.Trim(parts[i+1],
                "(")
                break
            }
            if p == "TABLE" && i+1 < len(parts) &&
parts[i+1] != "IF" {
                currentTable = strings.Trim(parts[i+1],
                "(")
                break
            }
        }
        if currentTable != "" {
            tables[currentTable] =
make(map[string]string)
        }
        continue
    }

    // Detect CREATE TABLE end
    if inCreateTable && strings.HasPrefix(trimmed,
    ");") {
        inCreateTable = false
        currentTable = ""
        continue
    }

    // Parse column definitions
    if inCreateTable && currentTable != "" {
        // Skip constraints and indexes
        if strings.HasPrefix(trimmed, "CONSTRAINT") ||
strings.HasPrefix(trimmed, "PRIMARY KEY") ||
strings.HasPrefix(trimmed, "CREATE INDEX")
||
strings.HasPrefix(trimmed, "UNIQUE") ||
strings.HasPrefix(trimmed, "CHECK") ||

```

```

        strings.HasPrefix(trimmed, "FOREIGN KEY") ||
        trimmed == "" {
            continue
        }

        // Parse column: name TYPE ...
        colParts := strings.Fields(trimmed)
        if len(colParts) >= 2 {
            colName := strings.Trim(colParts[0], ",")
            colType := colParts[1]
            tables[currentTable][colName] = colType
        }
    }
}

return tables
}

```

Test Command:

```

go test -race ./internal/schema/... -run
    TestSchemaDriftGuard -v

```

Acceptance Criteria: - Test passes with zero differences between primary schema and migration chain - If differences exist, they must be resolved before this item is closed - Test must run in CI (when CI is enabled in Phase 9)

Migration Required: If drift is detected, regenerate 0001 primary schema.sql using migrations/postgresql/.gen_schema.py or fix the migration chain to match.

PH0-002: SWIM Stop/Leave Double-Close (F1)

Priority: P0 — CRITICAL

Concurrency Hazard ID: F1

Severity: Major / Confidence: High

Constitution Reference: §7.1 (quality guarantee)

Estimated Effort: 4 person-hours

Status: Not Started

Problem Statement:

The SWIM protocol's `Stop()` and `Leave()` methods both close the `p.stopCh` channel. If `Stop()` is called followed by `Leave()` (or vice versa), or if `Stop()` is called twice, the program panics with a “close of closed channel” error. This is a production crash scenario during shutdown sequences.

Root Cause Analysis:

```
// CURRENT CODE (pkg/swim/protocol.go:692 – approximate)
func (p *Protocol) Stop() {
    close(p.stopCh) // PANIC if called twice or after
                    Leave()
    p.cancel()
}

func (p *Protocol) Leave() {
    close(p.stopCh) // PANIC if called twice or after
                    Stop()
    // ... graceful leave logic
}
```

The channel `p.stopCh` is shared between two methods with no coordination. In a shutdown scenario where a signal handler calls `Stop()` and a deferred cleanup calls `Leave()`, the double-close is inevitable.

File: `pkg/swim/protocol.go`

Implementation Specification:

```
// Add to Protocol struct:
type Protocol struct {
    // ... existing fields ...
    stopCh      chan struct{}
    closeOnce   sync.Once // NEW: protects channel close
    cancel      context.CancelFunc
    wg          sync.WaitGroup // NEW: tracks goroutines
}

// Modify Stop() to use sync.Once:
func (p *Protocol) Stop() {
    p.closeOnce.Do(func() {
        close(p.stopCh)
    })
    p.cancel()
    p.wg.Wait() // Wait for all goroutines to finish
}

// Modify Leave() to use the same sync.Once:
func (p *Protocol) Leave() {
    p.closeOnce.Do(func() {
        close(p.stopCh)
    })
    // ... graceful leave logic (broadcast leave message,
    // etc.)
}
```



```

    p.cancel()
    p.wg.Wait()
}

```

Test File: pkg/swim/protocol_test.go (add test)

```

func TestStopOnce(t *testing.T) {
    p := NewProtocol(Config{
        NodeID:      "test-node",
        BindAddr:    "127.0.0.1:0",
        GossipInterval: 100 * time.Millisecond,
    })

    // Start the protocol
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()
    go p.Run(ctx)

    // Call Stop twice – must not panic
    p.Stop()
    p.Stop() // Second call must be a no-op

    // Call Leave after Stop – must not panic
    p.Leave() // Must be a no-op since stopCh already
              // closed
}

```

Test Command:

```

go test -race ./pkg/swim/... -run TestStopOnce -v -count=100

```

The `-count=100` flag runs the test 100 times to detect intermittent race conditions that might not manifest in a single run.

Acceptance Criteria: - Test passes with zero panics across 100 iterations
 - `go test -race ./pkg/swim/...` passes with zero race reports -
 Double-close scenario is impossible regardless of call ordering

Additional Considerations:

The `Leave()` method should also broadcast a leave message to other cluster members before closing. The `sync.Once` must wrap only the channel close, not the leave message broadcast:

```

func (p *Protocol) Leave() {
    // First, try to broadcast leave message (best-effort)
    // This happens BEFORE the sync.Once in case the channel
    // is needed
}

```

```

    // for the broadcast.
    p.broadcastLeave()

    // Then ensure channel is closed exactly once
    p.closeOnce.Do(func() {
        close(p.stopCh)
    })
    p.cancel()
    p.wg.Wait()
}

```

PH0-003: SWIM HealthyMembers Data Race (F2)

Priority: P0 — CRITICAL

Concurrency Hazard ID: F2

Severity: Major / Confidence: High

Constitution Reference: §7.1 (quality guarantee)

Estimated Effort: 6 person-hours

Status: Not Started

Problem Statement:

The `HealthyMembers()` and `Members()` methods in `pkg/swim/protocol.go` read `Member.State` without synchronization, while concurrent goroutines update `Member.State` in the probe and suspicion handlers. This is a textbook data race that `go test -race` detects reliably.

Root Cause Analysis:

```

// CURRENT CODE (pkg/swim/protocol.go:447 – approximate)
type Member struct {
    ID      string
    State   MemberState // ACCESSED WITHOUT SYNCHRONIZATION
    Address string
    // ...
}

func (p *Protocol) HealthyMembers() []Member {
    var healthy []Member
    for _, m := range p.members {
        if m.State == StateAlive { // RACE: concurrent
            write in probe handler
            healthy = append(healthy, m)
        }
    }
    return healthy
}

```

```
// Concurrent writer:
func (p *Protocol) handleProbeResponse(resp *ProbeResponse)
{
    p.members[resp.From].State = StateAlive // RACE:
        concurrent read in HealthyMembers
}
```

File: pkg/swim/protocol.go , pkg/swim/member.go

Implementation Specification — Option A: Atomic Operations (Preferred for simple state):

```
type Member struct {
    ID      string
    state   int32 // accessed atomically; stores
        MemberState as int32
    Address string
    // ...
}

func (m *Member) GetState() MemberState {
    return MemberState(atomic.LoadInt32(&m.state))
}

func (m *Member) SetState(s MemberState) {
    atomic.StoreInt32(&m.state, int32(s))
}

// Update HealthyMembers to use atomic access:
func (p *Protocol) HealthyMembers() []Member {
    var healthy []Member
    for _, m := range p.members {
        if m.GetState() == StateAlive { // ATOMIC READ
            healthy = append(healthy, m)
        }
    }
    return healthy
}
```

Implementation Specification — Option B: RWMutex (If Member has multiple fields that must be read atomically):

```
type Member struct {
    mu      sync.RWMutex
    ID      string
    state   MemberState
    Address string
}
```

```

// ...
}

func (m *Member) GetState() MemberState {
    m.mu.RLock()
    defer m.mu.RUnlock()
    return m.state
}

func (m *Member) SetState(s MemberState) {
    m.mu.Lock()
    defer m.mu.Unlock()
    m.state = s
}

```

Recommendation: Use Option A (atomic operations) for `State` since it's a single integer field. Use Option B only if multiple fields must be read atomically (e.g., `State` + `LastSeen` + `Incarnation` as a consistent snapshot).

Test File: `pkg/swim/protocol_test.go` (add test)

```

func TestHealthyMembersRace(t *testing.T) {
    p := NewProtocol(Config{
        NodeID:      "test-node",
        BindAddr:     "127.0.0.1:0",
        GossipInterval: 10 * time.Millisecond,
    })

    // Add some members
    for i := 0; i < 10; i++ {
        p.AddMember(Member{
            ID:      fmt.Sprintf("member-%d", i),
            State:    StateAlive,
            Address:  fmt.Sprintf("127.0.0.1:%d", 5000+i),
        })
    }

    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()
    go p.Run(ctx)

    // Concurrent reader: continuously call HealthyMembers
    var wg sync.WaitGroup
    wg.Add(2)

    go func() {
        defer wg.Done()
        for i := 0; i < 10000; i++ {

```

```

        _ = p.HealthyMembers()
    }
}()

// Concurrent writer: continuously update member states
go func() {
    defer wg.Done()
    for i := 0; i < 10000; i++ {
        idx := i % 10
        p.SetMemberState(fmt.Sprintf("member-%d", idx),
            StateSuspect)
        p.SetMemberState(fmt.Sprintf("member-%d", idx),
            StateAlive)
    }
}()

wg.Wait()
}

```

Test Command:

```

go test -race ./pkg/swim/... -run TestHealthyMembersRace -v
      -count=10

```

Acceptance Criteria: - Test passes with zero data race reports across 10 iterations - `go test -race ./pkg/swim/...` passes clean - All member state access uses atomic operations or RWMutex

PH0-004: SWIM Untracked Goroutine (F3)

Priority: P0 — HIGH

Concurrency Hazard ID: F3

Severity: Minor / Confidence: High

Constitution Reference: §7.1 (quality guarantee)

Estimated Effort: 3 person-hours

Status: Not Started

Problem Statement:

The `probeRandomMember()` function is launched as a goroutine without being tracked by a `sync.WaitGroup`. When `Stop()` is called, the protocol waits for goroutines using `p.wg.Wait()`, but `probeRandomMember` goroutines are not in the `WaitGroup`. This causes: 1. Goroutine leaks (goroutines continue running after `Stop`) 2. Use-after-close (goroutine tries to send on closed channel) 3. Non-deterministic shutdown behavior

File: `pkg/swim/protocol.go:531`

Implementation Specification:

```
// CURRENT CODE:
go p.probeRandomMember()

// FIXED CODE:
p.wg.Add(1)
go func() {
    defer p.wg.Done()
    p.probeRandomMember()
}()
```

Apply this pattern to ALL goroutine launches in the SWIM protocol: - go p.probeRandomMember() - go p.handleSuspectTimeout() - go p.broadcastAlive() - go p.disseminate() - Any other go func() calls in the protocol

Test File: pkg/swim/protocol_test.go (add test)

```
func TestProbeGoroutineTracking(t *testing.T) {
    p := NewProtocol(Config{
        NodeID:      "test-node",
        BindAddr:     "127.0.0.1:0",
        GossipInterval: 50 * time.Millisecond,
    })

    // Add members to trigger probing
    for i := 0; i < 5; i++ {
        p.AddMember(Member{
            ID:      fmt.Sprintf("member-%d", i),
            State:   StateAlive,
            Address:  fmt.Sprintf("127.0.0.1:%d", 5000+i),
        })
    }

    ctx, cancel := context.WithCancel(context.Background())
    go p.Run(ctx)

    // Let it run for a few probe cycles
    time.Sleep(500 * time.Millisecond)

    // Stop and verify clean shutdown
    done := make(chan struct{})
    go func() {
        p.Stop()
        close(done)
    }()
}
```

```

select {
case <-done:
    // Success: Stop() returned, all goroutines cleaned
    up
case <-time.After(5 * time.Second):
    t.Fatal("Stop() did not return within 5 seconds –
    goroutine leak suspected")
}

// Verify no goroutines leaked by checking
    runtime.NumGoroutine()
// Note: this is a heuristic, not deterministic
runtime.GC()
time.Sleep(100 * time.Millisecond)
goroutinesAfter := runtime.NumGoroutine()
// We expect significantly fewer goroutines than during
    active probing
t.Logf("Goroutines after stop: %d", goroutinesAfter)
}

```

Test Command:

```

go test -race ./pkg/swim/... -run
    TestProbeGoroutineTracking -v

```

Acceptance Criteria: - `Stop()` returns within 5 seconds - No goroutine leaks detected - `go test -race ./pkg/swim/...` passes clean

PH0-005: SWIM Lock-Order Fragility (F4)

Priority: P0 — HIGH

Concurrency Hazard ID: F4

Severity: Major / Confidence: Medium

Constitution Reference: §7.1 (quality guarantee)

Estimated Effort: 6 person-hours

Status: Not Started

Problem Statement:

The failure detector's `Confirm()` method acquires `p.memberMu` and then `p.fdMu` in that order. Other methods may acquire them in the reverse order (`p.fdMu` then `p.memberMu`), creating a potential deadlock. While no deadlock has been observed in production, the lock ordering is not documented or enforced, making it fragile under code changes.

File: `pkg/swim/failure_detector.go:198`

Implementation Specification:

Step 1: Document the lock acquisition order.

Add a comment block at the top of `failure_detector.go`:

```
// LOCK ORDERING (MUST be respected throughout this
// package):
//
// 1. p.memberMu – protects the membership list
// 2. p.fdMu     – protects the failure detector state
//
// If both locks must be held, they MUST be acquired in this
// order.
// Violating this order WILL cause deadlocks.
//
// When only one lock is needed, acquire only that lock.
// When a method needs to release a lock before calling
// another method
// that may acquire it, use the pattern:
//   p.memberMu.Lock()
//   // ... read/modify members ...
//   p.memberMu.Unlock()
//   // Now safe to acquire fdMu
//   p.fdMu.Lock()
//   // ... read/modify failure detector ...
//   p.fdMu.Unlock()
```

Step 2: Add a lock-order assertion in debug builds.

```
// In a debug build, track which goroutine holds which lock
// using a lock-order tracker. This is a development-time
// tool
// that is compiled out in production builds.

//go:build debug

type lockTracker struct {
    mu      sync.Mutex
    holdings map[int64][]string // goroutine ID → list of
                          held locks
}

var globalLockTracker = &lockTracker{
    holdings: make(map[int64][]string),
}

func (lt *lockTracker) Acquire(lockName string) {
    gid := getGoroutineID()
    lt.mu.Lock()
```



```

defer lt.mu.Unlock()

held := lt.holdings[gid]

// Validate lock order: "memberMu" must come before
// "fdMu"
for _, existing := range held {
    if existing == "fdMu" && lockName == "memberMu" {
        panic(fmt.Sprintf("LOCK ORDER VIOLATION:
goroutine %d holds %q, attempting to acquire %q
(must be memberMu before fdMu)",
            gid, existing, lockName))
    }
}

lt.holdings[gid] = append(held, lockName)
}

func (lt *lockTracker) Release(lockName string) {
    gid := getGoroutineID()
    lt.mu.Lock()
    defer lt.mu.Unlock()

    held := lt.holdings[gid]
    for i, name := range held {
        if name == lockName {
            lt.holdings[gid] = append(held[:i],
                held[i+1:]...)
            return
        }
    }
}
}

```

Step 3: Audit all lock acquisitions in the SWIM package.

Manually review every `Lock()` and `RLock()` call in: - `pkg/swim/protocol.go` - `pkg/swim/failure_detector.go` - `pkg/swim/gossip.go` - `pkg/swim/member.go` - `pkg/swim/transport.go` - `pkg/swim/hierarchical.go`

For each acquisition, verify the lock order is respected. If any violation is found, restructure the code to release the first lock before acquiring the second, or restructure to avoid holding both simultaneously.

Test File: `pkg/swim/failure_detector_test.go` (add test)

```

func TestLockOrderConsistency(t *testing.T) {
    // Create a protocol with many members
    p := NewProtocol(Config{

```

```

        NodeID:      "test-node",
        BindAddr:    "127.0.0.1:0",
        GossipInterval: 50 * time.Millisecond,
    })

    for i := 0; i < 20; i++ {
        p.AddMember(Member{
            ID:      fmt.Sprintf("member-%d", i),
            State:    StateAlive,
            Address:  fmt.Sprintf("127.0.0.1:%d", 5000+i),
        })
    }

    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()
    go p.Run(ctx)

    // Stress test: concurrent operations that acquire
    // different locks
    var wg sync.WaitGroup
    for i := 0; i < 100; i++ {
        wg.Add(3)

        // Goroutine 1: Access members (acquires memberMu)
        go func() {
            defer wg.Done()
            _ = p.HealthyMembers()
            _ = p.Members()
        }()

        // Goroutine 2: Access failure detector (acquires
        // fdMu)
        go func() {
            defer wg.Done()
            _ = p.GetPhiAccrual("member-0")
            _ = p.GetFailureDetectorState()
        }()

        // Goroutine 3: Confirm failure (acquires both
        // locks)
        go func() {
            defer wg.Done()
            p.ConfirmFailure("member-1")
        }()
    }

    wg.Wait()
}

```

Test Command:

```
go test -race ./pkg/swim/... -run TestLockOrderConsistency
-v -count=50
```

Acceptance Criteria: - Lock ordering is documented in every file that acquires locks - Test passes with zero deadlocks across 50 iterations - `go test -race ./pkg/swim/...` passes clean - No lock-order violations in debug build testing

PH0-006: PTY Double-Close (F5)

Priority: P0 — CRITICAL

Concurrency Hazard ID: F5

Severity: Major / Confidence: High

Constitution Reference: §7.1 (quality guarantee)

Estimated Effort: 8 person-hours

Status: Not Started

Problem Statement:

The native PTY backend (`pkg/session/backends/native.go`) closes the PTY file descriptor in multiple code paths: 1. Session detach → close PTY 2. Session kill → close PTY 3. Process exit → close PTY 4. Cleanup goroutine → close PTY

If any two of these occur concurrently or in quick succession, the PTY fd is closed twice, causing a “use of closed file descriptor” error or EBADF from the kernel.

File: `pkg/session/backends/native.go`

Implementation Specification:

```
type NativeBackend struct {
    pty      *os.File
    closeOnce sync.Once // NEW: ensures PTY is closed
              exactly once
    // ... other fields ...
}

func (n *NativeBackend) Close() error {
    var err error
    n.closeOnce.Do(func() {
        if n.ptty != nil {
            err = n.ptty.Close()
        }
    })
    return err
}
```

```

}

// Update all close paths to use Close():
func (n *NativeBackend) Detach() error {
    // ... detach logic ...
    return n.Close() // Safe: sync.Once ensures single
                     // close
}

func (n *NativeBackend) Kill() error {
    // ... kill logic ...
    return n.Close() // Safe: sync.Once ensures single
                     // close
}

func (n *NativeBackend) handleProcessExit() {
    // ... cleanup logic ...
    n.Close() // Safe: sync.Once ensures single close
}

```

Additional Fix: Signal the read goroutine.

The read goroutine that reads from the PTY master fd must be signaled to stop when the PTY is closed:

```

type NativeBackend struct {
    pty          *os.File
    closeOnce    sync.Once
    done         chan struct{} // NEW: signals read goroutine
                // to stop
    wg           sync.WaitGroup // NEW: tracks read goroutine
    // ... other fields ...
}

func (n *NativeBackend) readLoop() {
    n.wg.Add(1)
    defer n.wg.Done()

    buf := make([]byte, 4096)
    for {
        select {
        case <-n.done:
            return
        default:
        }

        // Set a read deadline so we check done periodically
    }
}

```

```

        n.ptty.SetReadDeadline(time.Now().Add(100 *
            time.Millisecond))
        nr, err := n.ptty.Read(buf)
        if err != nil {
            if errors.Is(err, os.ErrDeadlineExceeded) {
                continue // Check done channel
            }
            if errors.Is(err, io.EOF) || errors.Is(err,
                os.ErrClosed) {
                return // PTY closed
            }
            // Log unexpected error
            continue
        }
        // Process read data ...
    }
}

func (n *NativeBackend) Close() error {
    var err error
    n.closeOnce.Do(func() {
        close(n.done) // Signal read goroutine to stop
        if n.ptty != nil {
            err = n.ptty.Close()
        }
        n.wg.Wait() // Wait for read goroutine to finish
    })
    return err
}

```

Test File: pkg/session/backends/native_test.go (add test)

```

func TestPTYCloseOnce(t *testing.T) {
    backend, err := NewNativeBackend(NativeConfig{
        Command: "echo",
        Args:     []string{"hello"},
    })
    require.NoError(t, err)

    // Start the backend
    err = backend.Start()
    require.NoError(t, err)

    // Close multiple times – must not panic or error
    err = backend.Close()
    assert.NoError(t, err)

    err = backend.Close() // Second close – must be a no-op
}

```

```

    assert.NoError(t, err)

    err = backend.Close() // Third close – must be a no-op
    assert.NoError(t, err)
}

func TestPTYConcurrentClose(t *testing.T) {
    backend, err := NewNativeBackend(NativeConfig{
        Command: "sleep",
        Args:     []string{"60"},
    })
    require.NoError(t, err)

    err = backend.Start()
    require.NoError(t, err)

    // Concurrent close from multiple goroutines
    var wg sync.WaitGroup
    for i := 0; i < 10; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            _ = backend.Close() // Must not panic
        }()
    }
    wg.Wait()
}

```

Test Command:

```

go test -race ./pkg/session/... -run TestPTYCloseOnce -v
go test -race ./pkg/session/... -run TestPTYConcurrentClose
-v

```

Acceptance Criteria: - Both tests pass with zero panics or errors - `go test -race ./pkg/session/...` passes clean - PTY is closed exactly once regardless of call pattern

PH0-007: Scheduler Unbounded Map (F6)

Priority: P0 — HIGH

Concurrency Hazard ID: F6

Severity: Major / Confidence: High

Constitution Reference: §7.1 (quality guarantee)

Estimated Effort: 6 person-hours

Status: Not Started

Problem Statement:

The scheduler's `placements` map (`pkg/scheduler/scheduler.go`) stores placement records for all jobs, but completed placements are never removed. Over time, this map grows without bound, consuming increasing memory. In a long-running scheduler, this is a memory leak that eventually causes OOM kills.

Root Cause Analysis:

```
// CURRENT CODE (pkg/scheduler/scheduler.go – approximate):
type Scheduler struct {
    placements map[string]*Placement // GROWS WITHOUT BOUND
    // ...
}

func (s *Scheduler) Schedule(req *ScheduleRequest)
    (*Placement, error) {
    p := &Placement{ID: uuid.New().String(), ...}
    s.placements[p.ID] = p // Added but never removed
    return p, nil
}

func (s *Scheduler) Complete(placementID string) {
    if p, ok := s.placements[placementID]; ok {
        p.Status = StatusCompleted
        // BUG: placement remains in map after completion
    }
}
```

File: `pkg/scheduler/scheduler.go`

Implementation Specification:

```
type Scheduler struct {
    placements      map[string]*Placement
    placementsMu    sync.RWMutex
    cleanupTicker   *time.Ticker
    cancelCleanup   context.CancelFunc
    // ... other fields ...
}

func (s *Scheduler) Start(ctx context.Context) error {
    ctx, s.cancelCleanup = context.WithCancel(ctx)

    // Start cleanup goroutine for completed placements
    s.cleanupTicker = time.NewTicker(5 * time.Minute)
    go s.cleanupPlacements(ctx)
```

```

    // ... rest of start logic ...
}

func (s *Scheduler) cleanupPlacements(ctx context.Context) {
    for {
        select {
        case <-ctx.Done():
            s.cleanupTicker.Stop()
            return
        case <-s.cleanupTicker.C:
            s.placementsMu.Lock()
            for id, p := range s.placements {
                if p.Status == StatusCompleted ||
                    p.Status == StatusFailed ||
                    p.Status == StatusCancelled {
                    delete(s.placements, id)
                }
            }
            s.placementsMu.Unlock()
        }
    }
}

func (s *Scheduler) Stop() {
    if s.cancelCleanup != nil {
        s.cancelCleanup()
    }
    // ... rest of stop logic ...
}

// Also remove placement immediately on completion:
func (s *Scheduler) Complete(placementID string) {
    s.placementsMu.Lock()
    defer s.placementsMu.Unlock()

    if p, ok := s.placements[placementID]; ok {
        p.Status = StatusCompleted
        // Remove immediately rather than waiting for
        // cleanup ticker
        delete(s.placements, placementID)
    }
}

```

Test File: pkg/scheduler/scheduler_test.go (add test)

```

func TestPlacementCleanup(t *testing.T) {
    sched := NewScheduler(Config{
        MaxPlacements: 1000,
    })
}

```



```

}))

ctx, cancel := context.WithCancel(context.Background())
defer cancel()

err := sched.Start(ctx)
require.NoError(t, err)

// Schedule and complete 1000 placements
for i := 0; i < 1000; i++ {
    p, err := sched.Schedule(&ScheduleRequest{
        JobName: fmt.Sprintf("job-%d", i),
        CPU:      1,
        Memory:   1024,
    })
    require.NoError(t, err)

    // Complete immediately
    sched.Complete(p.ID)
}

// Check that placements map is empty after completions
sched.placementsMu.RLock()
count := len(sched.placements)
sched.placementsMu.RUnlock()

assert.Equal(t, 0, count, "all completed placements
    should be removed from map")

// Schedule 100 placements without completing
for i := 0; i < 100; i++ {
    _, err := sched.Schedule(&ScheduleRequest{
        JobName: fmt.Sprintf("pending-job-%d", i),
        CPU:      1,
        Memory:   1024,
    })
    require.NoError(t, err)
}

// Check that pending placements are still in the map
sched.placementsMu.RLock()
count = len(sched.placements)
sched.placementsMu.RUnlock()

assert.Equal(t, 100, count, "pending placements should
    remain in map")
}

```

Test Command:

```
go test -race ./pkg/scheduler/... -run TestPlacementCleanup -v
```

Acceptance Criteria: - Completed placements are removed from the map immediately - Pending placements remain in the map - Memory usage does not grow unboundedly over time - `go test -race ./pkg/scheduler/...` passes clean

PH0-008: EventBus Unbounded Goroutines (F7)

Priority: P0 — HIGH

Concurrency Hazard ID: F7

Severity: Major / Confidence: High

Constitution Reference: §7.1 (quality guarantee)

Estimated Effort: 8 person-hours

Status: Not Started

Problem Statement:

The EventBus's `PublishAsync()` method spawns a new goroutine for each event delivery to each subscriber. Under high event rates (e.g., during cluster reconfiguration or health check storms), this creates an unbounded number of goroutines, leading to: 1. Memory exhaustion from goroutine stacks 2. Scheduler thrashing from too many runnable goroutines 3. Degraded latency as goroutines compete for CPU time

Root Cause Analysis:

```
// CURRENT CODE (EventBus/pkg/bus/bus.go – approximate):
func (b *Bus) PublishAsync(topic string, event Event) error {
    subscribers := b.getSubscribers(topic)
    for _, sub := range subscribers {
        go func(s Subscriber) { // UNBOUNDED GOROUTINE SPAWN
            s.Handle(event)
        }(sub)
    }
    return nil
}
```

If 100 events are published per second and each has 5 subscribers, that's 500 goroutines/second. If `Handle` takes 100ms, there will be 50 active goroutines at steady state. But if `Handle` slows down (e.g., due to downstream latency), goroutines accumulate without bound.

File: EventBus/pkg/bus/bus.go

Implementation Specification:

Replace the unbounded goroutine spawn with a bounded worker pool using pkg/semaphore :

```
type Bus struct {
    subscribers map[string][]Subscriber
    subMu       sync.RWMutex

    // Bounded worker pool
    workerPool chan struct{} // Semaphore-based bounded pool
    poolSize   int
    wg         sync.WaitGroup
    cancel     context.CancelFunc

    // ... other fields ...
}

func NewBus(config BusConfig) *Bus {
    poolSize := config.WorkerPoolSize
    if poolSize <= 0 {
        poolSize = runtime.NumCPU() * 4 // Default: 4x CPU cores
    }

    return &Bus{
        subscribers: make(map[string][]Subscriber),
        workerPool:  make(chan struct{}), poolSize),
        poolSize:    poolSize,
    }
}

func (b *Bus) Start(ctx context.Context) error {
    ctx, b.cancel = context.WithCancel(ctx)

    // Pre-fill worker pool semaphore
    for i := 0; i < b.poolSize; i++ {
        b.workerPool <- struct{}{}
    }

    return nil
}

func (b *Bus) PublishAsync(topic string, event Event) error {
    subscribers := b.getSubscribers(topic)
```

```

for _, sub := range subscribers {
    // Acquire a worker slot (blocks if pool is
    // exhausted)
    select {
    case <-b.workerPool:
        // Got a worker slot
    case <-ctx.Done():
        return ctx.Err()
    }

    b.wg.Add(1)
    go func(s Subscriber) {
        defer b.wg.Done()
        defer func() {
            b.workerPool <- struct{}
        }() // Return worker slot

        // Apply timeout to handler
        handlerCtx, cancel := context.WithTimeout(ctx,
30*time.Second)
        defer cancel()

        done := make(chan error, 1)
        go func() {
            done <- s.Handle(event)
        }()

        select {
        case <-handlerCtx.Done():
            // Handler timed out – log and continue
            log.Warnf("event handler timed out for
subscriber %T on topic %s", s, topic)
        case err := <-done:
            if err != nil {
                log.Errorf("event handler error for
subscriber %T on topic %s: %v", s, topic, err)
            }
        }
    }(sub)
}

return nil
}

func (b *Bus) Stop() {
    if b.cancel != nil {

```

```

        b.cancel()
    }
    b.wg.Wait() // Wait for all in-flight handlers to complete
}

```

Test File: EventBus/pkg/bus/bus_test.go (add test)

```

func TestBoundedPublish(t *testing.T) {
    bus := NewBus(BusConfig{
        WorkerPoolSize: 10, // Only 10 concurrent handlers
    })

    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    err := bus.Start(ctx)
    require.NoError(t, err)

    // Subscribe with a slow handler
    var activeHandlers int64
    slowHandler := &SlowHandler{
        duration: 100 * time.Millisecond,
        onActive: func() { atomic.AddInt64(&activeHandlers, 1) },
        onDone:   func() { atomic.AddInt64(&activeHandlers, -1) },
    }

    bus.Subscribe("test-topic", slowHandler)

    // Publish 100 events – should not exceed 10 concurrent handlers
    for i := 0; i < 100; i++ {
        err := bus.PublishAsync("test-topic", Event{Type: "test", Data: i})
        require.NoError(t, err)
    }

    // Wait for all events to be processed
    bus.wg.Wait()

    // The maximum concurrent handlers should never exceed poolSize
    maxActive := atomic.LoadInt64(&slowHandler.maxActive)
    assert.LessOrEqual(t, maxActive, int64(10),
        "concurrent handlers should not exceed pool size")
}

```

```

type SlowHandler struct {
    duration    time.Duration
    onActive    func()
    onDone      func()
    maxActive   int64
}

func (h *SlowHandler) Handle(event Event) error {
    h.onActive()
    current := atomic.AddInt64(&h.maxActive, 1)
    // Track maximum
    for {
        old := atomic.LoadInt64(&h.maxActive)
        if current >= old ||
            atomic.CompareAndSwapInt64(&h.maxActive, old,
            current) {
            break
        }
    }
    time.Sleep(h.duration)
    atomic.AddInt64(&h.maxActive, -1)
    h.onDone()
    return nil
}

```

Test Command:

```
go test -race ./EventBus/... -run TestBoundedPublish -v
```

Acceptance Criteria: - Concurrent handler count never exceeds pool size
 - No goroutine leaks after Stop() - Backpressure is applied when pool is exhausted - `go test -race ./EventBus/...` passes clean

PH0-009: Race Gate Full

Priority: P0 — GATE

Estimated Effort: 2 person-hours

Status: Not Started

Implementation:

After all F1-F7 fixes are applied, run the full race detector:

```
go test -race -count=1 ./...
```

This must pass with zero race reports. If any race is detected, it must be resolved before proceeding to Phase 1.

Procedure:

1. Run the race gate on a clean checkout
2. If any races are found, file a new HXC ticket for each
3. Fix each race before proceeding
4. Re-run the race gate
5. Repeat until zero races

Known Additional Potential Hazards (not yet confirmed):

ID	Location	Suspected Issue
F11	pkg/discovery/ tier_registry.go	Concurrent map access without full lock coverage
F12	pkg/events/ nats_backend.go	NATS connection state race
F13	internal/gpu/ manager.go	GPU state mutation during detection
F14	pkg/health/ rollup.go	Aggregation during concurrent health checks
F15	internal/messaging/ bus.go	Message delivery during bus shutdown

These will be investigated during the race gate. If any are confirmed, they are added to this phase.

Acceptance Criteria: - `go test -race -count=1 ./...` exits with code 0 - Zero race reports in the entire tree

PH0-010: Whole-Tree Build/Vet Gate

Priority: P0 — GATE

Estimated Effort: 1 person-hour

Status: Not Started

Implementation:

```
go build ./... && go vet ./...
```

Both must pass clean. If any build errors or vet warnings are found, they must be resolved before proceeding.

Common Vet Findings to Check:

1. `printf` : Incorrect format verbs in `fmt.Printf` family
2. `unreachable` : Code after `return/panic`
3. `copylocks` : Copying locks by value
4. `assign` : Useless assignments
5. `atomic` : Incorrect atomic usage
6. `structtag` : Malformed struct tags
7. `httpresponse` : Unchecked HTTP response body close

Procedure:

1. Run `go build ./...` — must succeed with zero errors
2. Run `go vet ./...` — must succeed with zero warnings
3. If any issues found, fix and re-run
4. Document any non-obvious vet suppressions

Acceptance Criteria: - `go build ./...` exits with code 0 - `go vet ./...` exits with code 0 - No warnings in either output

Phase 1: Dead-Code Resolution

Duration: 4–6 weeks

Owner: OWNER-GATED (requires owner triage decisions)

Prerequisite: Phase 0 complete (zero races, zero build failures)

Gate: All 178 orphaned packages triaged, Tier 1 wired

Estimated Effort: ~480 person-hours

Objective

Resolve the 178/212 orphaned `pkg/` packages by triaging each into WIRE-IN, PRUNE, or DOCUMENT-AS-LIBRARY categories, then wiring the Tier 1 packages into the appropriate `cmd/` binaries. This is the single largest structural gap in the project and directly addresses the CLAUDE-1 violation (implemented features that cannot be used by end users).

Entry Criteria

- ☐ Phase 0 exit criteria met
- ☐ All concurrency hazards resolved
- ☐ Race gate and build/vet gate passing

Exit Criteria

- ☐ All 178 orphaned packages triaged (WIRE-IN / PRUNE / DOCUMENT-AS-LIBRARY)
 - ☐ All Tier 1 packages wired into appropriate binaries
 - ☐ `make gate-check` passes (orphan-prevention gates)
 - ☐ No new race conditions introduced
 - ☐ Each wired package has at least one integration test
-

Wiring Tiers

The wiring tiers define the priority order for connecting orphaned packages to binaries. The order is determined by: 1. **Architectural criticality** — packages in the control-plane spine are most critical 2. **Dependency depth** — packages with many dependents should be wired first 3. **End-user visibility** — packages that directly enable user-facing features

Tier 1: Control-Plane Spine (HIGHEST PRIORITY — Wire-In)

These packages form the backbone of the distributed system. Without them, the cluster cannot provide consensus, leader election, distributed locking, or observability.

#	Package	LOC	Binary Target	Wiring Type	PH Item
1	<code>pkg/events</code>	~600	<code>cmd/helix-scheduler</code> , <code>cmd/helix-session</code>	Wire-in	PH1-001
2	<code>pkg/leader</code>	~200	<code>cmd/helix-scheduler</code>	Wire-in	PH1-002
3	<code>pkg/lock</code>	~150	<code>internal/node</code>	Wire-in	PH1-003
4	<code>pkg/tracing</code>	~300	All <code>cmd/binaries</code>	Wire-in	PH1-004
5	<code>pkg/hxcregistry</code>	~200		Wire-in	PH1-005

#	Package	LOC	Binary Target	Wiring Type	PH Item
			cmd/helix-gate		
6	pkg/etcd	~150	internal/node, cmd/helix-scheduler	Wire-in	PH1-006
7	pkg/crdt	~800	pkg/session (enhance)	Wire-in	PH1-007
8	pkg/deltacrdt	~400	pkg/session (enhance)	Wire-in	PH1-008
9	pkg/mvcc	~300	pkg/crdt (dependency)	Wire-in	PH1-009
10	pkg/raftprofile	~100	cmd/helix-scheduler	Wire-in	PH1-010
11	pkg/splitbrain	~200	pkg/swim (enhance)	Wire-in	PH1-011
12	pkg/splitbrainalert	~100	pkg/swim (enhance)	Wire-in	PH1-012
13	pkg/stonith	~400	cmd/helix-node	Wire-in	PH1-013

PH1-001: Wire pkg/events into cmd/helix-scheduler and cmd/helix-session

Priority: P0

Estimated Effort: 16 person-hours

Status: Not Started

Problem:

The event bus (pkg/events) is fully implemented with NATS and Helix backends, Avro wire format, and stream configuration — but it has zero importers from any binary. The scheduler and session services should publish domain events (e.g., WorkloadScheduled , SessionCreated , SessionDetached) for other services to consume.

Implementation Plan:

1. Add event bus initialization to cmd/helix-scheduler/main.go :

```

// In cmd/helix-scheduler/main.go
func main() {
    cfg := scheduler.LoadConfig()

    // Initialize event bus
    eventBus, err := events.NewBus(events.BusConfig{
        Backend:    events.BackendNATS, // or
               events.BackendHelix
        NATSURL:   cfg.NATSURL,
        Namespace: "helix.scheduler",
    })
    if err != nil {
        log.Fatalf("failed to create event bus: %v", err)
    }
    defer eventBus.Close()

    // Subscribe to relevant events
    eventBus.Subscribe("helix.node.*",
        &scheduler.NodeEventHandler{Scheduler: sched})

    // Wire event bus into scheduler
    sched.SetEventBus(eventBus)

    // ... rest of main
}

```

1. Publish domain events from the scheduler:

```

// In internal/scheduler/server.go
func (s *Server) Schedule(ctx context.Context, req
    *pb.ScheduleRequest) (*pb.ScheduleResponse, error) {
    // ... existing scheduling logic ...

    // Publish event after successful scheduling
    s.eventBus.Publish(ctx,
        "helix.scheduler.workload_scheduled", events.Event{
            Type:      "WorkloadScheduled",
            Timestamp: time.Now(),
            Payload:  map[string]interface{}{
                "workloadId": resp.WorkloadId,
                "nodeId":     resp.NodeId,
                "cpu":        req.Cpu,
                "memory":     req.Memory,
            },
        },
    )
}

```

```
    return resp, nil
}
```

1. Add event bus initialization to `cmd/helix-session/main.go` :

```
// In cmd/helix-session/main.go
func main() {
    cfg := session.LoadConfig()

    eventBus, err := events.NewBus(events.BusConfig{
        Backend:    events.BackendNATS,
        NATSURL:    cfg.NATSURL,
        Namespace:  "helix.session",
    })
    if err != nil {
        log.Fatalf("failed to create event bus: %v", err)
    }
    defer eventBus.Close()

    // Subscribe to scheduling events that affect sessions
    eventBus.Subscribe("helix.scheduler.workload_scheduled",
        &session.SchedulingHandler{Server: srv})

    // ... rest of main
}
```

Test:

```
func TestSchedulerEventPublishing(t *testing.T) {
    // Start scheduler with event bus
    bus := events.NewBus(events.BusConfig{Backend:
        events.BackendHelix})
    sched := scheduler.NewServer(bus)

    // Subscribe to scheduling events
    received := make(chan events.Event, 1)
    bus.Subscribe("helix.scheduler.workload_scheduled",
        &TestHandler{ch: received})

    // Schedule a workload
    resp, err := sched.Schedule(context.Background(),
        &pb.ScheduleRequest{...})
    require.NoError(t, err)

    // Verify event was published
    select {
    case event := <-received:
```

```

    assert.Equal(t, "WorkloadScheduled", event.Type)
    assert.Equal(t, resp.WorkloadId,
        event.Payload["workloadId"])
    case <-time.After(5 * time.Second):
        t.Fatal("event not received within timeout")
    }
}

```

Test Command:

```

go test -race ./cmd/helix-scheduler/... -run
    TestSchedulerEventPublishing -v
go test -race ./cmd/helix-session/... -run
    TestSessionEventSubscribing -v

```

Acceptance Criteria: - pkg/events is imported by cmd/helix-scheduler and cmd/helix-session - Scheduler publishes WorkloadScheduled events - Session service subscribes to scheduling events - Integration test proves event delivery end-to-end

PH1-002: Wire pkg/leader into cmd/helix-scheduler for leader election

Priority: P0

Estimated Effort: 12 person-hours

Status: Not Started

Problem:

The pkg/leader package implements etcd-based leader election with fencing tokens, but it has zero importers. The scheduler service needs leader election to ensure only one scheduler instance makes binding decisions at a time (or, in the Omega model, to coordinate multiple schedulers).

Anti-Bluff Note: The pkg/leader package has a PASS-bluff — the Election struct uses an int32 atomic flag instead of etcd. The real etcd implementation exists in etcd_election.go but is not the default. This item includes fixing the PASS-bluff by making the etcd implementation the default.

Implementation Plan:

1. Make etcd election the default (not the in-memory atomic flag)
2. Wire leader election into the scheduler's startup sequence
3. Only the leader scheduler performs binding decisions
4. Followers watch for leadership changes

```

// In cmd/helix-scheduler/main.go
func main() {
    cfg := scheduler.LoadConfig()

    // Initialize etcd client
    etcdClient, err := clientv3.New(clientv3.Config{
        Endpoints:    cfg.EtcdEndpoints,
        DialTimeout: 5 * time.Second,
    })
    if err != nil {
        log.Fatalf("failed to connect to etcd: %v", err)
    }
    defer etcdClient.Close()

    // Initialize leader election
    election, err :=
        leader.NewEtcdElection(leader.EtcdElectionConfig{
            Client:    etcdClient,
            Key:        "/helix/leader/scheduler",
            NodeID:     cfg.NodeID,
            LeaseTTL:   10 * time.Second,
        })
    if err != nil {
        log.Fatalf("failed to create leader election: %v",
            err)
    }

    // Start leader election campaign
    go election.Campaign(context.Background())

    // Wire election into scheduler
    sched.SetLeaderElection(election)

    // Only leader performs scheduling
    go func() {
        for {
            if election.IsLeader() {
                sched.RunSchedulingLoop(context.Background())
                return
            }
            time.Sleep(1 * time.Second)
        }
    }()

    // ... rest of main
}

```

Test:

```
func TestSchedulerLeaderElection(t *testing.T) {
    // Start embedded etcd
    etcd := setupEmbeddedEtcd(t)
    defer etcd.Close()

    // Create two scheduler instances
    election1 :=
        leader.NewEtcdElection(leader.EtcdElectionConfig{
            Client: etcd.Client,
            Key:     "/helix/leader/scheduler",
            NodeID: "scheduler-1",
        })

    election2 :=
        leader.NewEtcdElection(leader.EtcdElectionConfig{
            Client: etcd.Client,
            Key:     "/helix/leader/scheduler",
            NodeID: "scheduler-2",
        })

    // Start election campaigns
    go election1.Campaign(context.Background())
    go election2.Campaign(context.Background())

    // Wait for leader to be elected
    time.Sleep(3 * time.Second)

    // Exactly one should be leader
    l1 := election1.IsLeader()
    l2 := election2.IsLeader()
    assert.True(t, l1 || l2,
        "at least one scheduler must be leader")
    assert.False(t, l1 && l2, "only one scheduler can be
        leader at a time")

    // Kill the leader
    if l1 {
        election1.Resign()
        time.Sleep(3 * time.Second)
        assert.True(t, election2.IsLeader(), "scheduler-2
            should become leader after scheduler-1 resigns")
    } else {
        election2.Resign()
        time.Sleep(3 * time.Second)
        assert.True(t, election1.IsLeader(), "scheduler-1
            should become leader after scheduler-2 resigns")
    }
}
```

```
}  
}
```

Test Command:

```
go test -race ./cmd/helix-scheduler/... -run  
TestSchedulerLeaderElection -v -tags=integration
```

Acceptance Criteria: - `pkg/leader` is imported by `cmd/helix-scheduler` - Etcd election is the default (not in-memory) - Only one scheduler instance is leader at any time - Leadership failover works when leader resigns or crashes

PH1-003: Wire `pkg/lock` into `internal/node` for distributed locking

Priority: P0

Estimated Effort: 8 person-hours

Status: Not Started

Problem:

The `pkg/lock` package implements distributed locking via etcd (with `EtcdLocker`), but it has zero importers. Node registration and resource allocation require distributed locks to prevent race conditions in concurrent registration or allocation scenarios.

Implementation Plan:

1. Wire `EtcdLocker` into `internal/node` for node registration locks
2. Use distributed lock during node registration to prevent duplicate registrations
3. Use distributed lock during GPU allocation to prevent double-allocation

```
// In internal/node/registry.go  
func (r *EtcdRegistry) Register(ctx context.Context, node  
    *Node) error {  
    // Acquire distributed lock for this node's registration  
    lockKey := fmt.Sprintf("/helix/locks/node-register/%s",  
        node.ID)  
    locker := lock.NewEtcdLocker(r.etcdClient, lockKey,  
        lock.EtcdLockConfig{  
            LeaseTTL: 10 * time.Second,  
        })  
  
    if err := locker.Lock(ctx); err != nil {  
        return fmt.Errorf("failed to acquire registration  
            lock: %w", err)  
    }
```



```

    }
    defer locker.Unlock(ctx)

    // Check if node is already registered
    existing, err := r.GetNode(ctx, node.ID)
    if err == nil && existing != nil {
        return ErrNodeAlreadyRegistered
    }

    // Register the node
    return r.putNode(ctx, node)
}

```

Test Command:

```

go test -race ./internal/node/... -run
    TestDistributedNodeRegistration -v -tags=integration

```

Acceptance Criteria: - pkg/lock is imported by internal/node -
 Concurrent node registration attempts are serialized - No duplicate node registrations under concurrent access

PH1-004: Wire pkg/tracing into all cmd/ binaries via gRPC interceptors

Priority: P0

Estimated Effort: 12 person-hours

Status: Not Started

Problem:

The pkg/tracing package implements W3C Trace Context propagation, gRPC tracing, and an OTEL exporter — but it has zero importers (the original stub in package.go returns hardcoded trace IDs). This means the entire observability stack is blind: there are no distributed traces flowing through the system.

Anti-Bluff Note: The pkg/tracing package has a PASS-bluff — the StartSpan function in package.go returns TraceID: "trace-1", SpanID: "span-1". The real W3C implementation exists in w3c.go and grpc.go, but the stub masks it. This item includes fixing the PASS-bluff.

Implementation Plan:

1. Remove hardcoded IDs from package.go
2. Ensure W3C/OTel path is the default
3. Wire tracing into gRPC server interceptors for all services
4. Wire tracing into gRPC client interceptors for inter-service calls

5. Add trace context propagation in the gateway

```
// In cmd/helix-scheduler/main.go
func main() {
    // Initialize tracing
    tracer, err := tracing.NewTracer(tracing.TracerConfig{
        ServiceName: "helix-scheduler",
        OTLPEndpoint: cfg.OTLPEndpoint,
        Sampler:      tracing.AlwaysSample(),
    })
    if err != nil {
        log.Fatalf("failed to initialize tracer: %v", err)
    }
    defer tracer.Shutdown(context.Background())

    // Create gRPC server with tracing interceptor
    grpcServer := grpc.NewServer(
        grpc.UnaryInterceptor(tracing.UnaryServerInterceptor(tracer)),
        grpc.StreamInterceptor(tracing.StreamServerInterceptor(tracer)),
    )

    // ... register services and start server ...
}
```

Test:

```
func TestTracingPropagation(t *testing.T) {
    tracer := tracing.NewTracer(tracing.TracerConfig{
        ServiceName: "test-service",
        OTLPEndpoint: "localhost:4317",
    })

    // Create a span
    ctx, span := tracer.Start(context.Background(), "test-operation")
    defer span.End()

    // Verify trace ID is not hardcoded
    traceID := span.SpanContext().TraceID().String()
    assert.NotEqual(t, "trace-1", traceID, "trace ID must not be hardcoded")
    assert.NotEmpty(t, traceID, "trace ID must not be empty")

    // Verify span ID is not hardcoded
    spanID := span.SpanContext().SpanID().String()
    assert.NotEqual(t, "span-1", spanID, "span ID must not be hardcoded")
}
```

```
    assert.NotEmpty(t, spanID, "span ID must not be empty")
}
```

Test Command:

```
go test -race ./pkg/tracing/... -run TestTracingPropagation
-v
```

Acceptance Criteria: - `pkg/tracing` is imported by all `cmd/` binaries - Hardcoded IDs removed - W3C Trace Context propagation works across services - gRPC interceptors capture traces

PH1-005: Wire `pkg/hxcregistry` into `cmd/helix-gate` for registry queries

Priority: P1

Estimated Effort: 6 person-hours

Status: Not Started

Implementation Plan:

1. Wire `pkg/hxcregistry` into `cmd/helix-gate` so the gate can query the HXC registry for ticket status
2. Add a `--ticket` flag to `helix-gate` that checks if a specific ticket is completed
3. Use registry queries to inform gate decisions (e.g., skip checks for features that are explicitly deferred)

Test Command:

```
go test -race ./cmd/helix-gate/... -run
TestGateRegistryIntegration -v
```

Acceptance Criteria: - `pkg/hxcregistry` is imported by `cmd/helix-gate` - Gate can query ticket status from the registry - Integration test proves gate decision is informed by registry data

PH1-006 through PH1-013: Remaining Tier 1 Packages

Each remaining Tier 1 item follows the same pattern:

1. **Identify the binary target** — which `cmd/` binary should import this package
2. **Create wiring code** — add initialization and dependency injection in the binary's `main.go`
3. **Write integration test** — prove the wired feature works end-to-end

4. Verify no new races — run `go test -race ./...`

Item	Package	Binary	Integration Test
PH1-006	pkg/etcd	internal/ node, cmd/ helix- scheduler	TestEtcdKeyManagement
PH1-007	pkg/crdt	pkg/session (enhance)	TestCRDTSessionState
PH1-008	pkg/deltacrdt	pkg/session (enhance)	TestDeltaCRDTSync
PH1-009	pkg/mvcc	pkg/crdt (dependency)	TestMVCCSnapshots
PH1-010	pkg/raftprofile	cmd/helix- scheduler	TestRaftProfiling
PH1-011	pkg/splitbrain	pkg/swim (enhance)	TestSplitBrainDetection
PH1-012	pkg/ splitbrainalert	pkg/swim (enhance)	TestSplitBrainAlerting
PH1-013	pkg/stonith	cmd/helix- node	TestSTONITHFencing

Tier 2: GPU Stack (HIGH PRIORITY — Wire-In)

#	Package	LOC	Binary Target	PH Item
14	pkg/gpuattest	~300	cmd/helix-scheduler	PH1-014
15	pkg/middleware	~100	internal/gateway	PH1-015
16	pkg/pool	~100	cmd/gpu-pool-manager	PH1-016
17	pkg/burst	~50	cmd/burst-controller	PH1-017

PH1-014: Wire pkg/gpuattest into cmd/helix-scheduler

Wire GPU attestation gates into the scheduler so that GPU workloads are only scheduled on nodes that have passed attestation (Proof of Valuable Work, seal verification, multi-GPU attestation, spot checks).

Implementation: - Add attestation check to scheduler's filter pipeline - Scheduler queries `pkg/gpuattest` before binding a GPU workload to a

node - Nodes without valid attestation are excluded from GPU scheduling candidates

Test: TestGPUAttestationGate — verify that workloads are not scheduled on unattested nodes.

PH1-015: Wire pkg/middleware into internal/gateway

Replace the no-op middleware in internal/gateway with the real logging middleware from pkg/middleware. This requires first fixing the PASS-bluff in pkg/middleware (currently a no-op).

Anti-Bluff Fix: Implement real request logging with method, path, status code, duration, and request ID.

PH1-016: Wire pkg/pool into cmd/gpu-pool-manager

Wire the resource pooling abstraction into the GPU pool manager for GPU resource pool management.

PH1-017: Wire pkg/burst into cmd/burst-controller

Wire burst management into the burst controller for capacity management.

Tier 3: Scheduler Helpers (HIGH PRIORITY — Wire-In)

#	Package	LOC	Binary Target	PH Item
18	pkg/backfill	~200	cmd/helix-scheduler	PH1-018
19	pkg/priorityqueue	~100	cmd/helix-scheduler	PH1-019
20	pkg/nodeselector	~100	cmd/helix-scheduler	PH1-020
21	pkg/jobadmit	~100	cmd/helix-scheduler	PH1-021
22	pkg/costsched	~150	cmd/helix-scheduler	PH1-022
23	pkg/latencysched	~150	cmd/helix-scheduler	PH1-023
24	pkg/rebalance	~100	cmd/helix-scheduler	PH1-024
25	pkg/smartrouter	~100	cmd/helix-scheduler	PH1-025
26	pkg/workloadrouter	~100	cmd/helix-scheduler	PH1-026

PH1-018: Wire pkg/backfill into cmd/helix-scheduler

Wire backfill scheduling into the scheduler's queue processing. Backfill allows lower-priority jobs to use resources that would otherwise be idle, as long as they complete before a higher-priority job would start.

Implementation:

```
// In internal/scheduler/server.go
func (s *Server) initSchedulingPlugins() {
    // Wire backfill scheduler
    s.backfill = backfill.NewScheduler(backfill.Config{
        MaxBackfillJobs:      100,
        LookaheadDuration:    30 * time.Minute,
        DefaultJobDuration:   5 * time.Minute,
    })

    // Wire into scheduling loop
    s.AddPostSchedulingHook(s.backfill.ProcessBackfillCandidates)
}
```

PH1-019 through PH1-026: Follow the same pattern — wire the package into the scheduler's plugin system, add integration test, verify no races.

Tier 4: Federation & Data-Plane (MEDIUM PRIORITY — Wire-In)

#	Package	LOC	Binary Target	PH Item
27	pkg/federation	~300	New cmd/helix-federation	PH1-027
28	pkg/fedtrust	~100	pkg/federation	PH1-028
29	pkg/spiffefed	~100	pkg/federation	PH1-029
30	pkg/cellmesh	~100	pkg/federation	PH1-030
31	pkg/dataplane	~200	internal/gateway	PH1-031
32	pkg/nattraversal	~100	pkg/wireguard	PH1-032

Tier 5: Security Extensions (HIGH PRIORITY — Wire-In)

Security extensions provide defense-in-depth for the cluster. Without them, the cluster operates with minimal security posture — no anti-cheat,

no attestation-based admission, no GPU verification, no audit proof, and no capability-based access control.

#	Package	LOC	Binary Target	PH Item	Triage
27	pkg/anticheat	~100	cmd/helix-gateway	PH1-027	WIRE-IN
28	pkg/attestadmit	~100	cmd/helix-scheduler	PH1-028	WIRE-IN
29	pkg/gravaladmit	~100	cmd/helix-scheduler	PH1-029	WIRE-IN
30	pkg/gravalverify	~100	cmd/helix-security	PH1-030	WIRE-IN
31	pkg/scan	~100	make security-scan	PH1-031	WIRE-IN
32	pkg/exportcontrol	~50	cmd/helix-build	PH1-032	DOCUMENT
33	pkg/imagepolicy	~100	cmd/helix-build	PH1-033	WIRE-IN
34	pkg/capability	~100	cmd/helix-security	PH1-034	WIRE-IN
35	pkg/tofu	~50	pkg/security	PH1-035	WIRE-IN
36	pkg/anonymize	~50	pkg/audit	PH1-036	WIRE-IN
37	pkg/auditproof	~50	internal/security	PH1-037	WIRE-IN
38	pkg/doublecrypt	~200	cmd/e2ee-proxy	PH1-038	WIRE-IN

PH1-027: Wire pkg/anticheat into cmd/helix-gateway

Wire anti-cheat token verification into the gateway's authentication middleware. When a client submits a request with an anti-cheat token header, the gateway verifies the token before allowing the request to proceed. This prevents unauthorized access from compromised clients.

Implementation:

```
// In internal/gateway/auth.go
func (m *AuthMiddleware) validateAntiCheatToken(r
    *http.Request) error {
    token := r.Header.Get("X-Anti-Cheat-Token")
    if token == "" {
        return nil // Anti-cheat is optional; not all
                   // clients send tokens
    }
    if err := anticheat.Verify(token); err != nil {
        return fmt.Errorf("anti-cheat token verification
                           failed: %w", err)
    }
    return nil
}
```

PH1-028: Wire pkg/attestadmit into cmd/helix-scheduler

Wire attestation-based admission into the scheduler's filter pipeline. Before a workload is scheduled onto a node, the scheduler verifies that the node has passed attestation. This ensures workloads only run on verified hardware.

PH1-029: Wire pkg/gravaladmit into cmd/helix-scheduler

Wire GraVal-based admission control into the scheduler. GraVal provides a proof-of-valuable-work system that ensures nodes contribute legitimate computational resources before being admitted to the cluster.

Tier 6: Marketplace & Economics (MEDIUM PRIORITY — Wire-In/Document)

These packages implement the dual-revenue model (HLX internal + TAO external marketplace). They are important for the project's economic sustainability but are not required for core cluster operation.

#	Package	LOC	Binary Target	PH Item	Triage
39	pkg/ marketplaceadapter	~200	New cmd/ helix- marketplace	PH1-039	WIRE- IN

#	Package	LOC	Binary Target	PH Item	Triage
40	pkg/revenueopt	~100	cmd/helix-scheduler	PH1-040	WIRE-IN
41	pkg/billingfsm	~200	cmd/helix-marketplace	PH1-041	WIRE-IN
42	pkg/metering	~100	pkg/metrics	PH1-042	WIRE-IN
43	pkg/costrouter	~100	cmd/helix-scheduler	PH1-043	WIRE-IN
44	pkg/costtracker	~100	pkg/metrics	PH1-044	WIRE-IN
45	pkg/tierdef	~100	internal/gateway	PH1-045	WIRE-IN

Tier 7: Infrastructure & Operations (MEDIUM PRIORITY — Wire-In)

#	Package	LOC	Binary Target	PH Item	Triage
46	pkg/cloudspot	~150	cmd/helix-agent	PH1-046	WIRE-IN
47	pkg/storage	~300	cmd/helix-build	PH1-047	WIRE-IN
48	pkg/gitops	~200	New cmd/helix-gitops	PH1-048	DOCUMENT
49	pkg/grafanadash	~200	pkg/metrics	PH1-049	WIRE-IN
50	pkg/watchtower	~100	cmd/helix-agent	PH1-050	WIRE-IN
51	pkg/watchmanager	~100	internal/node	PH1-051	WIRE-IN
52	pkg/heartbeatcoalescer	~100	pkg/swim	PH1-052	WIRE-IN

Tier 8: Caching & Performance (MEDIUM PRIORITY — Wire-In)

#	Package	LOC	Binary Target	PH Item	Triage
53	pkg/tieredcache	~100	internal/gateway	PH1-053	WIRE-IN
54	pkg/burstcapacity	~100	cmd/burst-controller	PH1-054	WIRE-IN
55	pkg/bursthysteresis	~100	cmd/burst-controller	PH1-055	WIRE-IN
56	pkg/thermalwarm	~50	pkg/scheduler	PH1-056	WIRE-IN
57	pkg/ewmarank	~50	pkg/scheduler	PH1-057	WIRE-IN
58	pkg/ringavg	~50	pkg/metrics	PH1-058	WIRE-IN

Tier 9: LLM & Inference (MEDIUM PRIORITY — Wire-In)

#	Package	LOC	Binary Target	PH Item	Triage
59	pkg/quantization	~200	cmd/helix-llm	PH1-059	WIRE-IN
60	pkg/chutes	~200	cmd/helix-llm	PH1-060	WIRE-IN
61	pkg/inferenceproxy	~200	internal/gateway	PH1-061	WIRE-IN
62	pkg/llmfailover	~50	cmd/helix-llm	PH1-062	WIRE-IN
63	pkg/modelintegrity	~100	cmd/helix-llm	PH1-063	WIRE-IN
64	pkg/modelretry	~50	cmd/helix-llm	PH1-064	WIRE-IN

Tier 10: Edge Computing (MEDIUM PRIORITY — Wire-In)

#	Package	LOC	Binary Target	PH Item	Triage
65	pkg/edge	~200	cmd/helix-agent	PH1-065	WIRE-IN
66	pkg/edgerefusion	~100	cmd/helix-agent	PH1-066	WIRE-IN
67	pkg/edgeheartbeat	~200	cmd/helix-agent	PH1-067	WIRE-IN
68	pkg/edgeverify	~50	cmd/helix-security	PH1-068	WIRE-IN

Tier 11: Testing Infrastructure (MEDIUM PRIORITY — Wire-In)

These packages are critical for the project's quality assurance but are currently orphaned — they exist but are not wired into any test runner or CI pipeline.

#	Package	LOC	Binary Target	PH Item	Triage
69	pkg/testing/dst	~200	cmd/dst-sim	PH1-069	WIRE-IN
70	pkg/testing/dstscale	~100	cmd/dst-sim	PH1-070	WIRE-IN
71	pkg/testing/dstcompress	~50	cmd/dst-sim	PH1-071	WIRE-IN
72	pkg/testing/dstworkload	~100	cmd/dst-sim	PH1-072	WIRE-IN
73	pkg/testing/chaos	~200	New cmd/helix-chaos	PH1-073	WIRE-IN

#	Package	LOC	Binary Target	PH Item	Triage
74	pkg/ testing/ evidence	~100	cmd/ helix- test	PH1-074	WIRE-IN
75	pkg/ testing/ scenario	~100	cmd/dst- sim	PH1-075	WIRE-IN
76	pkg/ testing/ runner	~100	cmd/ helix- test	PH1-076	WIRE-IN
77	pkg/ testing/ snapshot	~100	cmd/ helix- snapshot	PH1-077	WIRE-IN
78	pkg/ testing/ regression	~100	test/ scripts	PH1-078	WIRE-IN
79	pkg/ testing/ device	~200	cmd/ helix- test	PH1-079	WIRE-IN
80	pkg/ testing/ turmoil	~100	cmd/dst- sim	PH1-080	WIRE-IN
81	pkg/ testing/ instance	~100	cmd/ helix- test	PH1-081	WIRE-IN
82	pkg/ testing/ sessionfsm	~100	cmd/ helix- test	PH1-082	WIRE-IN
83	pkg/ porcupine	~100	cmd/dst- sim	PH1-083	WIRE-IN

Tier 12: Utility & Specialized (LOW PRIORITY — Document-As-Library or Prune)

These packages are utility libraries that may be useful but are not directly required by any binary. They should be documented as standalone libraries and pruned if they have no realistic integration path.

#	Package	LOC	Triage	Rationale
84	pkg/classads	~300	WIRE-IN	Used by scheduler for HTCSS-style job matching
85	pkg/offlinesync	~200	DOCUMENT	Standalone offline sync library
86	pkg/idempotent	~100	WIRE-IN	Used by all services for idempotent operations
87	pkg/failconfirm	~50	WIRE-IN	Used by SWIM for failure confirmation
88	pkg/fallbackchain	~50	WIRE-IN	Used by service mesh for fallback routing
89	pkg/headersanitize	~50	WIRE-IN	Used by gateway for header sanitization
90	pkg/fmea	~100	DOCUMENT	Standalone FMEA analysis tool
91	pkg/forecast	~100	WIRE-IN	Used by scheduler for resource forecasting
92	pkg/compliancedoc	~50	DOCUMENT	Compliance documentation generator
93	pkg/helixtask	~50	WIRE-IN	Task management utility
94	pkg/local	~50	DOCUMENT	Local execution mode (not for production)

#	Package	LOC	Triage	Rationale
95	pkg/passthrough	~50	PRUNE	No clear integration path
96	pkg/phase7matrix	~50	PRUNE	Phase-specific utility, no longer needed
97	pkg/hybridtco	~50	DOCUMENT	TCO calculation utility (library)
98	pkg/stressmark	~50	WIRE-IN	Stress test marking utility
99	pkg/hlc	~50	WIRE-IN	Hybrid logical clocks (CRDT dependency)
100	pkg/antientropy	~100	WIRE-IN	Anti-entropy sync (CRDT dependency)
101	pkg/semaphore	~50	WIRE-IN	Already used by EventBus (should be verified)
102	pkg/lru	~50	WIRE-IN	Used by tieredcache
103	pkg/pubsub	~100	WIRE-IN	Simple pub/sub (may overlap with events)
104	pkg/workerpool	~100	WIRE-IN	Bounded worker pool utility
105	pkg/serde	~200	WIRE-IN	Serialization/deserialization helpers
106	pkg/validator	~100	WIRE-IN	Input validation framework
107	pkg/context	~50	WIRE-IN	Context helpers

#	Package	LOC	Triage	Rationale
108	pkg/retry	~100	WIRE-IN	Retry with backoff
109	pkg/discovery	~300	WIRE-IN	Service discovery (already wired in gateway)

Remaining 70 packages (not individually enumerated in this plan) follow the same triage process. Each will be evaluated during Phase 1 execution and assigned WIRE-IN / PRUNE / DOCUMENT-AS-LIBRARY. The default triage for unevaluated packages is DOCUMENT-AS-LIBRARY — this is the safest default because it preserves the code while not requiring immediate integration effort.

Triage Execution Procedure

For each orphaned package, the following steps are performed:

1. **Dependency analysis:** Run `go list -f '{{.Importers}}' ./pkg/<name>/...` to identify any existing importers
2. **Interface audit:** Review exported types and functions to determine integration potential
3. **Anti-bluff check:** Verify the package has real (non-stub) implementation
4. **Constitution check:** Determine if wiring the package would violate any constitutional rule
5. **Triage decision:** Assign WIRE-IN / PRUNE / DOCUMENT-AS-LIBRARY
6. **Owner sign-off:** For WIRE-IN decisions, confirm with owner that the feature is desired
7. **Execution:** Wire-in, prune, or document the package
8. **Verification:** Run `make gate-check` to ensure no new orphans introduced

Pruning Decision Matrix

For each orphaned package, the following criteria determine the triage decision:

Criterion	WIRE-IN	PRUNE	DOCUMENT-AS-LIBRARY
Has importers after Tier 1 wiring?	Yes	No	No
	Yes	No	Partially

Criterion	WIRE-IN	PRUNE	DOCUMENT-AS-LIBRARY
Implements a feature claimed in the architecture?			
Has real (non-stub) implementation?	Yes	—	Yes
Has integration potential?	High	None	Medium
Anti-bluff status?	PASS	FAIL (stub)	Partial

Pruning Procedure:

1. Create a branch: `prune/<package-name>`
2. Remove the package directory
3. Run `go build ./...` to verify no remaining imports
4. Run `go test ./...` to verify no test failures
5. If any remaining imports, those importers must be updated first
6. Merge only after full-tree build and test pass

Document-As-Library Procedure:

1. Add a `README.md` to the package explaining its purpose and intended use
2. Add `// Package <name> is a standalone library for <purpose>. doc comment`
3. Register in `pkg/library registry.yaml`
4. Add an example in `examples/` if applicable

Phase 2: Test Coverage to Practical Maximum

Duration: 6–8 weeks

Owner: Autonomous

Prerequisite: Phase 1 Tier 1 wiring complete

Gate: All wired packages have integration tests

Estimated Effort: ~640 person-hours

Objective

Achieve maximum practical test coverage across all test types defined in the Test Strategy document (TS-100PCT). This phase creates the test infrastructure that Phases 3 (Challenges) and 6 (Bucket A) depend on.

Entry Criteria

- ☐ Phase 1 Tier 1 wiring complete
- ☐ All wired packages build and pass unit tests
- ☐ Race gate still passing

Exit Criteria

- ☐ All `cmd/` binaries have at least one test file
 - ☐ All security packages have fuzz tests
 - ☐ All `internal/` packages have integration tests
 - ☐ At least 2 E2E test suites passing
 - ☐ At least 2 chaos tests passing
 - ☐ Stress test framework operational
 - ☐ Mutation test count ≥ 50 (from 5)
 - ☐ Coverage gate (`pkg/covgate`) wired into `make gate-check`
-

PH2-001: `cmd/helix-gate/main_test.go`

Priority: P0

Estimated Effort: 8 person-hours

Currently has zero tests. The gate binary is critical for quality enforcement — it must be tested.

Test Functions:


```

{
    "e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b8",
    "abcdbcdecdefdefgefghfghighijhijkijkljklmklmnlmnomnopnopq",
    "248d6a61d20638b8e5c026930c3e6039a33ce45964ff2167f6ecedd419db06"
}

for _, v := range vectors {
    got := fmt.Sprintf("%x",
        crypto.Hash([]byte(v.input)))
    assert.Equal(t, v.expected, got, "SHA-256 KAT
        mismatch for input %q", v.input)
}
}

```

PH2-003: pkg/security/fuzz_test.go

Priority: P0

Estimated Effort: 6 person-hours

```
func FuzzTLSConfig(f *testing.F) {
    f.Add("localhost", "cert.pem", "key.pem")
    f.Fuzz(func(t *testing.T, host, certPath, keyPath
        string) {
        _, err := security.NewTLSConfig(host, certPath,
            keyPath)
        // Should not panic regardless of input
        _ = err
    })
}
```

PH2-004: pkg/hybridkex/fuzz_test.go

Priority: P0

Estimated Effort: 4 person-hours

Expand existing fuzz tests with additional seed inputs and boundary conditions for the ML-KEM-768 hybrid key exchange.

PH2-005: Integration test suite for pkg/etcd

Priority: P1

Estimated Effort: 12 person-hours

```
//go:build integration
```

```
func TestEtcdPutGet(t *testing.T) { /* ... */ }
func TestEtcdWatch(t *testing.T) { /* ... */ }
```

```
func TestEtcdLease(t *testing.T)      { /* ... */ }
func TestEtcdLock(t *testing.T)      { /* ... */ }
func TestEtcdTransaction(t *testing.T) { /* ... */ }
```

PH2-006: Integration test for pkg/lock/EtcdLocker

Priority: P1

Estimated Effort: 8 person-hours

Test distributed locking with concurrent lockers, lock expiration, and lock renewal.

PH2-007: E2E test for workload placement

Priority: P0

Estimated Effort: 16 person-hours

gateway → policy → scheduler → node

Full end-to-end test: submit a workload via the gateway API, verify it passes policy checks, gets scheduled by the scheduler, and is assigned to a node.

PH2-008: E2E test for session lifecycle

Priority: P0

Estimated Effort: 16 person-hours

create → attach → detach → resume → kill

Full session lifecycle test through the API.

PH2-009: Chaos test for SWIM partition and recovery

Priority: P1

Estimated Effort: 12 person-hours

Create a 5-node SWIM cluster, partition it, verify majority partition continues, heal partition, verify convergence.

PH2-010: Chaos test for etcd leader election failover

Priority: P1

Estimated Effort: 12 person-hours

Kill the etcd leader, verify a new leader is elected, verify the scheduler resumes operation.

PH2-011 through PH2-050: Additional Test Files

Each additional test file follows the same pattern: identify the package, determine the test type needed, write the test, verify it passes with `-race`.

Item	Test File	Test Functions
PH2-011	pkg/jwt/package_test.go	TestParseRealJWT , TestParse_Mutation
PH2-012	pkg/config/ package_test.go	TestLoadFromEnv , TestLoad_Mutation
PH2-013	pkg/middleware/ logging_test.go	TestLoggingMiddlewareReal , TestLoggingMiddleware_Mutation
PH2-014	pkg/websocket/ upgrade_test.go	TestUpgradeReal , TestUpgrade_Mutation
PH2-015	pkg/infra/ integration_test.go	TestDockerBoot
PH2-016	pkg/leader/ integration_test.go	TestEtcdElection
PH2-017	pkg/crdt/merge_test.go	TestGCounterMerge , TestORSetMerge
PH2-018	pkg/deltacrdt/ delta_test.go	TestDeltaPropagation
PH2-019	pkg/mvcc/ snapshot_test.go	TestMVCCSnapshot , TestMVCCConcurrentReads
PH2-020	pkg/splitbrain/ detect_test.go	TestSplitBrainDetection , TestSplitBrainClassification
PH2-021	pkg/stonith/ fence_test.go	TestIPMIFencing , TestSBDFencing
PH2-022	pkg/discovery/ tier_registry_test.go	TestTierRegistry , TestTierRegistryRace
PH2-023	pkg/events/avro_test.go	TestAvroWireFormat , TestAvroRoundTrip
PH2-024	pkg/events/ nats_backend_test.go	TestNATSBackend
PH2-025	pkg/wireguard/ mesh_test.go	TestMeshEstablishment

Item	Test File	Test Functions
PH2-026	pkg/wireguard/ keyrotation_test.go	TestKeyRotation
PH2-027	pkg/wireguard/ holepunch_test.go	TestNATHolePunch
PH2-028	pkg/scheduler/ plugin_test.go	TestBackfillPlugin, TestPriorityQueuePlugin
PH2-029	pkg/scheduler/ gang_test.go	TestGangScheduling, TestGangPreemption
PH2-030	pkg/session/ crdt_checkpoint_test.go	TestCRDTCheckpoint, TestCRDTCheckpoint_Mutation
PH2-031	pkg/session/ migration_test.go	TestSessionMigration
PH2-032	pkg/gpuattest/ povw_test.go	TestPoVWAttestation
PH2-033	pkg/gpuattest/ seal_test.go	TestSealVerification
PH2-034	pkg/gpuattest/ multigpu_test.go	TestMultiGPUAttestation
PH2-035	pkg/gpuattest/ spotcheck_test.go	TestSpotCheckAttestation
PH2-036	internal/gateway/ auth_test.go	TestRealJWTAUTH
PH2-037	internal/security/ jwt_test.go	TestRealJWTSigning
PH2-038	internal/health/ grpc_test.go	TestGRPCHealthWatch
PH2-039	internal/build/ orchestrator_test.go	TestOrchestratorPath
PH2-040	pkg/swim/ chaos_partition_test.go	TestSWIMPartitionRecovery
PH2-041	pkg/health/ rollup_test.go	TestRollupAggregation
PH2-042	pkg/ratelimit/ sliding_test.go	TestSlidingWindowRateLimit

Item	Test File	Test Functions
PH2-043	pkg/tieredcache/ lru_test.go	TestLRUEviction
PH2-044	pkg/burstcapacity/ calc_test.go	TestBurstCapacityCalculation
PH2-045	pkg/bursthysteresis/ logic_test.go	TestBurstHysteresis
PH2-046	pkg/backoff/ exponential_test.go	TestExponentialBackoff , TestBackoff_Mutation
PH2-047	pkg/retry/ with_backoff_test.go	TestRetryWithBackoff
PH2-048	pkg/semaphore/ weighted_test.go	TestWeightedSemaphore
PH2-049	pkg/errors/ multi_test.go	TestMultiError , TestMultiError_Mutation
PH2-050	pkg/log/ structured_test.go	TestStructuredLogging , TestStructuredLogging_Mutation

PH2-051: Frontend Tests

Priority: P1

Estimated Effort: 40 person-hours

The web frontend (web/) has zero test coverage. Create test files for:

File	Test Framework	Test Functions
web/src/App.test.tsx	Jest + React Testing Library	AppRenders , AppRoutesCorrectly
web/src/pages/ Cluster.test.tsx	Jest	ClusterPageRenders , NodeListDisplays
web/src/pages/ Sessions.test.tsx	Jest	SessionsPageRenders , SessionCreateWorks
web/src/pages/ Jobs.test.tsx	Jest	JobsPageRenders , JobSubmitWorks
web/src/pages/ Builds.test.tsx	Jest	BuildsPageRenders
	Jest	HealthPageRenders

File	Test Framework	Test Functions
web/src/pages/Health.test.tsx		
web/src/pages/Security.test.tsx	Jest	SecurityPageRenders
web/src/api/client.test.ts	Jest	ClientAuth , ClientGet , ClientPost
web/src/components/NodeCard.test.tsx	Jest	NodeCardRenders , NodeCardStatus
web/src/components/SessionCard.test.tsx	Jest	SessionCardRenders
web/src/components/JobCard.test.tsx	Jest	JobCardRenders
web/src/components/BuildCard.test.tsx	Jest	BuildCardRenders
web/src/layout/Header.test.tsx	Jest	HeaderRenders , NavigationWorks
web/src/layout/Sidebar.test.tsx	Jest	SidebarRenders , SidebarNavigation
web/src/layout/Footer.test.tsx	Jest	FooterRenders

PH2-052: Stress Test Suite (48-hour sustained load)

Priority: P1

Estimated Effort: 24 person-hours

Create a stress test framework that runs sustained load for 48 hours:

```
// test/stress/stress_test.go
//go:build stress

func TestSustainedScheduling(t *testing.T)    { /* 48h of
    scheduling requests */ }
func TestSustainedSessionCreation(t *testing.T) { /* 48h of
    session create/destroy */ }
```



```

func TestMemoryStability(t *testing.T)           { /* verify
    no memory leaks */ }
func TestGoroutineStability(t *testing.T)        { /* verify
    no goroutine leaks */ }

```

Infrastructure: - Kubernetes job that runs the stress test - Prometheus metrics collection during the test - Alert on memory/goroutine growth exceeding thresholds - Automatic report generation at test completion

PH2-053: Soak Test Plan (7-day GPU fleet)

Priority: P2

Estimated Effort: 16 person-hours

Create a soak test plan for 7-day sustained operation with GPU workloads:

1. Deploy 5 GPU nodes
2. Run continuous inference workloads for 7 days
3. Monitor: memory leaks, goroutine leaks, GPU memory fragmentation, thermal throttling
4. Verify: no OOM kills, no GPU faults, no scheduling failures
5. Collect metrics for regression analysis

PH2-054: Mutation Test Expansion

Priority: P1

Estimated Effort: 24 person-hours

Expand mutation tests from 5 cases to 50+ cases. Create a mutation test template:

```

// Template for mutation test
func TestX_Mutation(t *testing.T) {
    // Step 1: Run the original test to establish baseline
    originalResult := runTestX(t)
    require.NoError(t, originalResult)

    // Step 2: Apply mutation
    applyMutationX()
    defer revertMutationX()

    // Step 3: Run the test again – it should now FAIL
    mutatedResult := runTestX(t)
    assert.Error(t, mutatedResult,

```

```
"mutation: test should fail when implementation is  
broken")  
}
```

Target packages for mutation tests (priority order):

1. pkg/jwt — TestParse_Mutation (tampered token should fail)
 2. pkg/crypto — TestHash_Mutation (wrong hash should fail)
 3. pkg/config — TestLoad_Mutation (wrong config should fail)
 4. pkg/backoff — TestDuration_Mutation (cap removal should fail)
 5. pkg/events — TestPublish_Mutation (event loss should be detected)
 6. pkg/ratelimit — TestRateLimit_Mutation (limit bypass should fail)
 7. pkg/semaphore — TestAcquire_Mutation (over-acquisition should fail)
 8. pkg/errors — TestMultiError_Mutation (error loss should fail)
 9. pkg/log — TestStructuredLogging_Mutation (field loss should fail)
 10. pkg/discovery — TestTierRegistry_Mutation (registration loss should fail)
-

Phase 3: Challenges Integration

Duration: 4-6 weeks

Owner: Autonomous

Prerequisite: Phase 2 test infrastructure operational

Gate: Challenge bank populated, evidence pipeline operational

Estimated Effort: ~320 person-hours

Objective

Integrate the HelixQA Challenges framework into the project, populate the challenge bank with helix_cluster-specific challenges, and establish the anti-bluff and evidence collection pipelines.

PH3-001: Add challenges submodule to go.work

Priority: P0

Estimated Effort: 2 person-hours

```
git submodule add https://github.com/HelixDevelopment/  
    challenges.git challenges  
git submodule update --init --recursive
```

Update go.mod :

```
replace github.com/HelixDevelopment/challenges => ./
challenges
```

Update go.work :

```
use ./challenges
```

PH3-002: Create banks/helix_cluster/ YAML definitions (30+ challenges)

Priority: P0

Estimated Effort: 80 person-hours

Create YAML challenge definitions for all major features:

Category	Challenges
Node Orchestration	node-register , node-heartbeat , node-gpu-report , node-deregister
Scheduling	scheduler-schedule , scheduler-preempt , scheduler-backfill , scheduler-gang
Session Management	session-create , session-attach , session-detach , session-resume , session-kill
Health Monitoring	health-check , health-watch , health-aggregate
Security	security-auth , security-rbac , security-mtls
Build Service	build-submit , build-stream , build-cancel , build-go , build-podman
GPU Management	gpu-allocate , gpu-release , gpu-mig , gpu-attest
Networking	wireguard-mesh , wireguard-nat-traversal
Federation	federation-register , federation-workload-migration

PH3-003: Wire challenge runner into Makefile

Priority: P1

Estimated Effort: 4 person-hours

```
.PHONY: challenges
challenges:
    go run ./cmd/helix-test run --bank=helix_cluster --
    evidence-dir=qa-results/challenges

.PHONY: challenges-quick
challenges-quick:
    go run ./cmd/helix-test run --bank=helix_cluster --
    difficulty=easy,medium --timeout=5m
```

PH3-004: Create shell script challenges for infrastructure

Priority: P1

Estimated Effort: 16 person-hours

Create shell-based challenges that verify infrastructure operations:

```
#!/bin/bash
# challenges/shell/node-register.sh
set -euo pipefail

ENDPOINT="${HELIX_ENDPOINT:-localhost:50053}"
EVIDENCE_DIR="${EVIDENCE_DIR:-./evidence/node-register}"
mkdir -p "$EVIDENCE_DIR"

RESPONSE=$(grpcurl -plaintext -d '{
    "hostname": "challenge-node",
    "ip_address": "10.0.0.100",
    "port": 50053,
    "labels": {"type": "challenge"},
    "gpu_count": 0
}' "$ENDPOINT" helix.v1.NodeService/Register)

echo "$RESPONSE" > "$EVIDENCE_DIR/response.json"

NODE_ID=$(echo "$RESPONSE" | jq -r '.nodeId')
if [ -z "$NODE_ID" ] || [ "$NODE_ID" = "null" ]; then
    echo "FAIL: Node ID is empty"
    exit 1
fi
```

```
echo "PASS: Node registered with ID $NODE_ID"
```

PH3-005: Create userflow challenges for API/gRPC

Priority: P1

Estimated Effort: 24 person-hours

Create multi-step userflow challenges that test complete API workflows:

```
# challenges/userflows/complete-job-lifecycle.yaml
name: Complete Job Lifecycle
description: Submit, monitor, and retrieve a job result
steps:
  - name: authenticate
    action: grpc_call
    service: SecurityService
    method: Authenticate
    input: { username: "test-user", password: "test-
      password" }
    assertions:
      - jsonpath_exists: "$.token"
    save: { token: "$.token" }

  - name: submit-job
    action: grpc_call
    service: SchedulerService
    method: Schedule
    headers: { authorization: "Bearer {{token}}" }
    input: { jobName: "test-job", resourceRequirements: {
      cpuCores: 2, memoryMb: 1024 } }
    assertions:
      - jsonpath_exists: "$.jobId"
    save: { jobId: "$.jobId" }

  - name: verify-scheduled
    action: grpc_call
    service: SchedulerService
    method: GetJobStatus
    input: { jobId: "{{jobId}}" }
    assertions:
      - jsonpath_equals: { path: "$.status", value:
        "SCHEDULED" }

  - name: verify-running
    action: wait_until
    timeout: 30s
    check:
```

```
action: grpc_call
service: SchedulerService
method: GetJobStatus
input: { jobId: "{{jobId}}" }
assertions:
  - jsonpath_in: { path: "$.status", values:
    ["RUNNING", "COMPLETED"] }
```

PH3-006: Anti-bluff scanner integration

Priority: P0

Estimated Effort: 16 person-hours

Integrate the bluff scanner into the challenge runner to detect PASS-bluffs automatically:

```
// In challenges/runner/anti_bluff.go
type BluffScanner struct {
    rules []BluffRule
}

var DefaultBluffRules = []BluffRule{
    {ID: "BR-01", Name: "NoStatusCodeOnly", Check:
        checkNoStatusCodeOnly},
    {ID: "BR-02", Name: "NoHardcodedExpected", Check:
        checkNoHardcodedExpected},
    {ID: "BR-03", Name: "NoMockTarget", Check:
        checkNoMockTarget},
    {ID: "BR-04", Name: "MissingNegativeTest", Check:
        checkMissingNegativeTest},
    {ID: "BR-05", Name: "MissingEvidence", Check:
        checkMissingEvidence},
    {ID: "BR-06", Name: "ShallowAssertion", Check:
        checkShallowAssertion},
    {ID: "BR-07", Name: "DeterministicBypass", Check:
        checkDeterministicBypass},
    {ID: "BR-08", Name: "MissingSinkVerification", Check:
        checkMissingSinkVerification},
}
```

PH3-007: Mutation testing integration

Priority: P1

Estimated Effort: 16 person-hours

Wire mutation testing into the challenge runner so that each challenge can verify its underlying tests catch intentional breakage.

PH3-008: Evidence collection pipeline

Priority: P0

Estimated Effort: 16 person-hours

Implement the evidence collection pipeline per Constitution §11.4.2:

```
qa-results/
├── challenges/<run-id>/
│   ├── node-register/
│   │   ├── result.json
│   │   ├── response.json
│   │   └── etcd state.json
│   ├── scheduler-schedule/
│   │   ├── result.json
│   │   └── schedule_response.json
│   └── summary.json
├── coverage/<run-id>/
├── benchmarks/<run-id>/
├── security/<run-id>/
├── chaos/<run-id>/
└── compliance/<run-id>/
```

Phase 4: Responsiveness

Duration: 2-3 weeks

Owner: Autonomous

Prerequisite: Phase 0 complete (can run in parallel with Phases 2-3)

Gate: No unbounded resources, no blocking on cold paths

Estimated Effort: ~80 person-hours

Objective

Fix the remaining concurrency hazards (F8-F10) and audit the entire codebase for lazy-initialization and bounded-pool anti-patterns.

PH4-001: F8 fix — EventBus Subscribe buffered delivery

Priority: P1

Concurrency Hazard ID: F8

Estimated Effort: 4 person-hours

Problem: The EventBus's `Subscribe` method creates unbuffered delivery channels. When a subscriber is slow, the publisher blocks, creating backpressure that stalls the entire event pipeline.

Fix:

```
// In EventBus/pkg/bus/bus.go
func (b *Bus) Subscribe(topic string, handler Subscriber)
    error {
    // Use a buffered channel to prevent publisher blocking
    ch := make(chan Event,
        b.config.SubscriberBufferSize) // Default: 256
        events
    // ...
}
```

Add a non-blocking send with drop logging:

```
func (b *Bus) deliver(sub Subscriber, event Event) {
    select {
    case sub.ch <- event:
        // Delivered successfully
    default:
        // Channel full – drop event and log
        b.metrics.EventsDropped.Inc()
        log.Warnf("event dropped for subscriber %T on topic
            %s: channel full",
                sub, event.Topic)
    }
}
```

PH4-002: F9 fix — TieredCache hot tier size cap

Priority: P1

Concurrency Hazard ID: F9

Estimated Effort: 4 person-hours

Problem: The `TieredCache` hot tier has no size cap, allowing unbounded memory growth.

Fix:

```
// In pkg/tieredcache/tieredcache.go
type TieredCache struct {
    hotTier    *lru.Cache // Add LRU eviction
    coldTier    redis.Client
    maxSize     int        // Maximum items in hot tier
    // ...
}

func NewTieredCache(config TieredCacheConfig) *TieredCache {
```



```

maxSize := config.HotTierMaxSize
if maxSize <= 0 {
    maxSize = 10000 // Default: 10,000 items
}

lruCache, _ := lru.New(maxSize) // LRU with size cap

return &TieredCache{
    hotTier: lruCache,
    coldTier: config.RedisClient,
    maxSize: maxSize,
}
}

```

PH4-003: F10 fix — Per-event timer allocation optimization

Priority: P2

Concurrency Hazard ID: F10

Estimated Effort: 6 person-hours

Problem: The EventBus creates a new `time.NewTimer` for each event delivery, causing GC pressure under high event rates.

Fix:

Replace per-event timer allocation with a shared timer pool:

```

var timerPool = sync.Pool{
    New: func() interface{} {
        return time.NewTimer(0)
    },
}

func (b *Bus) trySend(ch chan Event, event Event, timeout
    time.Duration) error {
    timer := timerPool.Get().(*time.Timer)
    defer timerPool.Put(timer)

    timer.Reset(timeout)
    select {
    case ch <- event:
        return nil
    case <-timer.C:
        return ErrDeliveryTimeout
    }
}

```

PH4-004: Tree-wide lazy-init audit

Priority: P1

Estimated Effort: 12 person-hours

Audit the entire codebase for lazy-initialization anti-patterns where expensive operations are performed on the first request path:

Checklist: 1. Search for `sync.Once` patterns that may block request handling 2. Search for `init()` functions that perform I/O 3. Search for global variables that are initialized on first use 4. Ensure all initialization is done during startup, not during request handling

PH4-005: Tree-wide bounded pool audit

Priority: P1

Estimated Effort: 12 person-hours

Audit the entire codebase for unbounded goroutine pools:

Checklist: 1. Search for `go func()` without a bounded worker pool 2. Search for `go func()` without WaitGroup tracking 3. Search for `make(chan struct{})` without capacity limit 4. Ensure all goroutine pools have configurable size limits 5. Ensure all goroutine pools have graceful shutdown

Phase 5: Security Scanning Deep

Duration: 2-3 weeks

Owner: Autonomous

Prerequisite: Phase 0 complete (can run in parallel with Phases 2-3)

Gate: gosec + govulncheck + trivy in make security-scan

Estimated Effort: ~60 person-hours

Objective

Promote security scanning from the current single-tool approach (govulncheck only) to a comprehensive four-tool pipeline, and wire it into the build gates.

PH5-001: Promote gosec to make security-scan

Priority: P0

Estimated Effort: 8 person-hours

Current State: gosec is available but findings don't block the build.

Implementation:

```
# In Makefile
.PHONY: security-scan
security-scan: govulncheck gosec trivy
    @echo "Security scan complete"

.PHONY: gosec
gosec:
    gosec -fmt=json -out=qa-results/security/gosec.json
    -severity=high ./...
    @if [ -s qa-results/security/gosec.json ]; then \
        echo "ERROR: gosec found high-severity
        issues"; \
        exit 1; \
    fi

.PHONY: govulncheck
govulncheck:
    govulncheck ./... > qa-results/security/
    govulncheck.txt 2>&1
    @if grep -q "Vulnerability" qa-results/security/
    govulncheck.txt; then \
        echo "ERROR: govulncheck found
        vulnerabilities"; \
        exit 1; \
    fi
```

PH5-002: Wire govulncheck into CI

Priority: P1 (requires Phase 9 CI re-enablement)

Estimated Effort: 4 person-hours

Create a GitHub Actions workflow for govulncheck (stored in `.github/workflows/disabled/` until Phase 9 re-enables CI):

```
# .github/workflows/disabled/security-scan.yml
name: Security Scan
on: [push, pull_request]
jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
```

```
with: { go-version: '1.25' }  
- run: make security-scan
```

PH5-003: Trivy scan integration

Priority: P1

Estimated Effort: 12 person-hours

Wire Trivy container image scanning into the security pipeline:

```
.PHONY: trivy  
trivy:  
    trivy image --format json --output qa-results/  
    security/trivy.json helix-cluster:latest  
    trivy image --exit-code 1 --severity HIGH,CRITICAL  
    helix-cluster:latest
```

PH5-004: SonarQube setup (when tokens available)

Priority: P2 (blocked on SONAR_TOKEN)

Estimated Effort: 8 person-hours

The `deploy/compose/security_sonarqube.yml` file exists but requires `SONAR_TOKEN`. When the owner provides the token:

1. Start SonarQube: `docker compose -f deploy/compose/security_sonarqube.yml up -d`
2. Configure project in SonarQube
3. Add sonar-scanner to CI pipeline
4. Set quality gate to enforce zero new bugs/vulnerabilities

Phase 6: Bucket A Registry Backlog

Duration: 8-12 weeks

Owner: Autonomous

Prerequisite: Phases 1-5 complete

Gate: All 78 Bucket A items resolved

Estimated Effort: ~960 person-hours

Objective

Resolve the 78 autonomously-actionable items from the HXC registry. These are items that do not require owner decisions, hardware access, or cloud resources.

Bucket A Item Categories

Category	Count	Example Items
CRITICAL fixes	3	CRIT-1 (JWT), CRIT-2 (health gRPC), CRIT-3 (build imports)
Stub bluff replacements	8	pkg/config, pkg/grpccutil, pkg/infra, pkg/jwt, pkg/leader, pkg/middleware, pkg/tracing, pkg/websocket
Wiring completions	15	Gateway routing, helixd service orchestration
Test additions	20	Missing test files, fuzz tests, integration tests
Security fixes	10	Token validation, RBAC enforcement, mTLS completion
Documentation updates	10	Stale docs, missing API docs, schema drift
Performance fixes	5	Lazy init, bounded pools, cache caps
Cross-platform fixes	7	macOS equivalents, Windows stubs

Detailed Bucket A Item Registry

The following table enumerates all 78 Bucket A items with HXC references, file paths, estimated effort, and priority:

CRITICAL Fixes (3 items)

#	HXC	Description	File	Effort	Priority
BA-001	HXC-1213	Replace JWT stub tokens with real verification	pkg/jwt/package.go , internal/security/server.go	16h	P0

#	HXC	Description	File	Effort	Priority
BA-002	—	Add gRPC health service to cmd/helix-health	internal/health/server.go	12h	P0
BA-003	—	Fix build service import paths	cmd/helix-build/main.go	8h	P0

Stub Bluff Replacements (8 items)

#	HXC	Description	File	Bluff Type	Effort
BA-004	—	pkg/config: implement real env/file loading	pkg/config/package.go	Hardcoded defaults	8h
BA-005	—	pkg/grpcutil: implement real interceptors	pkg/grpcutil/interceptors.go	No-op functions	8h
BA-006	—	pkg/infra: implement real Docker/Podman backend	pkg/infra/orchestrator.go	Simulation only	16h
BA-007	—	pkg/jwt: implement real JWT verification	pkg/jwt/package.go	String splitting only	12h
BA-008	—	pkg/leader: make etcd election the default	pkg/leader/etcd_election.go	Atomic flag only	6h
BA-009	—	pkg/middleware: implement real logging middleware	pkg/middleware/logging.go	No-op middleware	8h
BA-010	—	pkg/tracing: remove hardcoded	pkg/tracing/package.go	Hardcoded "trace-1"	8h

#	HXC	Description	File	Bluff Type	Effort
		IDs, use W3C/OTel			
BA-011	—	pkg/websocket: implement real WebSocket upgrade	pkg/websocket/upgrade.go	Nil return	8h

Wiring Completions (15 items)

#	HXC	Description	Target Binary	Effort
BA-012	—	Complete gateway routing for all 14 services	cmd/helix-gateway	16h
BA-013	—	Implement helixd service orchestration	cmd/helixd	24h
BA-014	—	Wire pkg/discovery into gateway for service discovery	internal/gateway	8h
BA-015	—	Wire pkg/ratelimit into gateway middleware	internal/gateway	4h
BA-016	—	Wire pkg/metrics into all services	All cmd/binaries	12h
BA-017	—	Wire pkg/events into remaining services	cmd/helix-node, cmd/helix-health, cmd/helix-build	12h
BA-018	—			6h

#	HXC	Description	Target Binary	Effort
		Wire pkg/ health gRPC into gateway	internal/ gateway	
BA-019	—	Wire internal/gpu into cmd/ gpu-pool- manager	cmd/gpu- pool- manager	8h
BA-020	—	Wire pkg/ session backends into cmd/ htmux	cmd/htmux	8h
BA-021	—	Wire pkg/ wireguard into cmd/ helix-agent	cmd/helix- agent	12h
BA-022	—	Wire pkg/ security into cmd/e2ee- proxy	cmd/e2ee- proxy	6h
BA-023	—	Wire internal/ policy into scheduler pipeline	internal/ scheduler	8h
BA-024	—	Wire pkg/ storage into build service	internal/ build	6h
BA-025	—	Wire pkg/ grafanadash into monitoring pipeline	New cmd/ helix- monitor	8h
BA-026	—	Wire pkg/ forecast into scheduler	internal/ scheduler	4h

Test Additions (20 items)

#	HXC	Description	Test File	Effort
BA-027	—	pkg/ anticheat token verification tests	pkg/anticheat/token_test.go	4h
BA-028	—	pkg/ attestadmit admission control tests	pkg/attestadmit/admit_test.go	4h
BA-029	—	pkg/ gpuattest attestation integration tests	pkg/gpuattest/ attest_integration_test.go	8h
BA-030	—	pkg/crdt merge conflict resolution tests	pkg/crdt/merge_test.go	6h
BA-031	—	pkg/ splitbrain detection and classification tests	pkg/splitbrain/ classification_test.go	6h
BA-032	—	pkg/stonith fencing integration tests	pkg/stonith/ fence_integration_test.go	8h
BA-033	—	pkg/ wireguard key rotation integration tests	pkg/wireguard/ keyrotation_test.go	6h
BA-034	—	pkg/ federation cross-cell migration tests	pkg/federation/ migration_test.go	8h

#	HXC	Description	Test File	Effort
BA-035	—	internal/ gateway auth flow E2E test	internal/gateway/ auth_e2e_test.go	8h
BA-036	—	cmd/helix- build orchestrator path test	internal/build/ orchestrator_test.go	6h
BA-037	—	cmd/helix- scheduler gang scheduling test	internal/scheduler/ gang_test.go	6h
BA-038	—	pkg/events Avro round- trip fuzz test	pkg/events/avro_fuzz_test.go	4h
BA-039	—	pkg/classads expression parser fuzz test	pkg/classads/ parser_fuzz_test.go	4h
BA-040	—	pkg/ wireguard config parsing fuzz test	pkg/wireguard/ config_fuzz_test.go	4h
BA-041	—	pkg/swim protocol message fuzz test	pkg/swim/ protocol_fuzz_test.go	4h
BA-042	—	internal/ security SPIFFE CA integration test	internal/security/ spiffe_ca_integration_test.go	8h
BA-043	—	internal/ schema migration chain integration test	internal/schema/ migration_integration_test.go	8h
BA-044	—			16h

#	HXC	Description	Test File	Effort
		Cross-service workload placement E2E test	test/e2e/workload_placement_test.go	
BA-045	—	Session lifecycle E2E test	test/e2e/session_lifecycle_test.go	16h
BA-046	—	SWIM partition recovery chaos test	test/chaos/swim_partition_test.go	12h

Security Fixes (10 items)

#	HXC	Description	File	Effort
BA-047	—	Token validation requires real signature check	pkg/jwt/package.go	12h
BA-048	—	RBAC scope enforcement at gateway level	internal/gateway/auth.go	8h
BA-049	—	mTLS e2e completion for all service pairs	All cmd/ binaries	16h
BA-050	—	SPIFFE CA integration with all services	All cmd/ binaries	12h
BA-051	—	Secret injection from Vault for all services	pkg/security/secret_injector.go	8h
BA-052	—	Audit log trigger verification	internal/schema/audit_test.go	4h

#	HXC	Description	File	Effort
BA-053	—	Export control compliance check	pkg/exportcontrol/compliance_test.go	4h
BA-054	—	Image policy enforcement in build service	pkg/imagepolicy/policy_test.go	6h
BA-055	—	Anti-cheat token verification integration	pkg/anticheat/integration_test.go	6h
BA-056	—	Double-encryption envelope verification	pkg/doublecrypt/verify_test.go	4h

Documentation Updates (10 items)

#	HXC	Description	Target	Effort
BA-057	—	Update main README with current architecture	README.md	4h
BA-058	—	Complete OpenAPI specification	internal/gateway/openapi.yaml	8h
BA-059	—	Fix SQL schema documentation drift	docs/sql/	4h
BA-060	—	Create API user guide	docs/api/UserGuide.md	8h
BA-061	—	Create deployment runbook	docs/ops/DeploymentRunbook.md	6h
BA-062	—	Create incident response runbook	docs/ops/IncidentResponse.md	6h
BA-063	—	Register all docs in docs_chain contexts	.docs_chain/contexts/	4h
BA-064	—	Generate architecture diagrams from code	docs/diagrams/	8h
BA-065	—		CHANGELOG.md	4h

#	HXC	Description	Target	Effort
		Create changelog from HXC registry		
BA-066	—	Verify docs_chain export pipeline (MD→PDF→HTML→DOCX)	scripts/docs/export.sh	8h

Performance Fixes (5 items)

#	HXC	Description	File	Effort
BA-067	—	Lazy initialization audit for all cmd/ binaries	All cmd/main.go files	8h
BA-068	—	Bounded pool audit for all goroutine spawns	Tree-wide	8h
BA-069	—	TieredCache hot tier size cap (F9)	pkg/tieredcache/tieredcache.go	4h
BA-070	—	EventBus buffered delivery (F8)	EventBus/pkg/bus/bus.go	4h
BA-071	—	Per-event timer allocation optimization (F10)	EventBus/pkg/bus/bus.go	6h

Cross-Platform Fixes (7 items)

#	HXC	Description	File	Effort
BA-072	—	Replace pkg/resources/proc_mock.go with real proc_darwin.go	pkg/resources/	4h
BA-073	—			8h

#	HXC	Description	File	Effort
		Implement real macOS GPU detection via IOKit	pkg/resources/drm_darwin.go	
BA-074	—	Implement real macOS accelerator detection	pkg/resources/accel_darwin.go	6h
BA-075	—	Add Windows/WSL2 implementations for key packages	pkg/resources/proc_windows.go	16h
BA-076	—	Fix WireGuard test skips on macOS/non-root	pkg/wireguard/*_test.go	6h
BA-077	—	Add boot coordination for macOS	internal/console/darwin_boot.go	8h
BA-078	—	Cross-platform session backend testing	pkg/session/backends/	8h

CRIT-1: Replace JWT Stub Tokens with Real Verification

Priority: P0 — CRITICAL

HXC Reference: HXC-1213

Estimated Effort: 16 person-hours

Current State: pkg/jwt only splits JWT strings on . without verification. ValidateToken RPC accepts any token format.

Implementation:

1. Replace pkg/jwt with github.com/golang-jwt/jwt/v5
2. Add signing key management (from Vault or SPIFFE)
3. Enforce token expiration
4. Add claims validation (issuer, audience, subject)
5. Wire RBAC scopes into token claims

```
// pkg/jwt/package.go (REWRITE)
package jwt

import (
```

```

    "github.com/golang-jwt/jwt/v5"
)

type Token struct {
    Raw      string
    Claims   jwt.MapClaims
    Header   map[string]interface{}
    Signature string
}

func Parse(tokenString string, verifyKey interface{})
    (*Token, error) {
    parsed, err := jwt.Parse(tokenString, func(token
        *jwt.Token) (interface{}, error) {
        // Validate signing method
        if _, ok := token.Method.(*jwt.SigningMethodHMAC); !
            ok {
            return nil, fmt.Errorf("unexpected signing
                method: %v", token.Header["alg"])
            }
            return verifyKey, nil
        })
    if err != nil {
        return nil, fmt.Errorf("token validation failed:
            %w", err)
    }

    claims, ok := parsed.Claims.(jwt.MapClaims)
    if !ok {
        return nil, fmt.Errorf("invalid claims type")
    }

    return &Token{
        Raw:      parsed.Raw,
        Claims:   claims,
        Header:   parsed.Header,
        Signature: parsed.Signature,
    }, nil
}

```

CRIT-2: Add gRPC Health Service to cmd/helix-health

Priority: P0 — CRITICAL

Estimated Effort: 12 person-hours

Current State: Health service serves HTTP-only health checks without gRPC integration. The gateway cannot use gRPC-based health checking for

backend services, which means routing decisions are made without real health information.

Implementation:

1. Implement `grpc.health.v1.HealthServer` interface in `internal/health/server.go`
2. Add a gRPC server alongside the existing HTTP server
3. Register the health service with the gRPC server
4. Implement the `Watch` streaming RPC for real-time health updates
5. Connect health events to the event bus for cross-service notification

```
// In internal/health/grpc_server.go (NEW)
package health

import (
    "context"
    "sync"

    "google.golang.org/grpc/health/grpc_health_v1"
)

type grpcHealthServer struct {
    grpc_health_v1.UnimplementedHealthServer
    mu          sync.RWMutex
    services
        map[string]grpc_health_v1.HealthCheckResponse_ServingStatus
    watchers map[string][]chan
        grpc_health_v1.HealthCheckResponse_ServingStatus
}

func NewGRPCHealthServer() *grpcHealthServer {
    return &grpcHealthServer{
        services:
            make(map[string]grpc_health_v1.HealthCheckResponse_ServingStatus),
        watchers: make(map[string][]chan
            grpc_health_v1.HealthCheckResponse_ServingStatus),
    }
}

func (s *grpcHealthServer) Check(ctx context.Context, req
    *grpc_health_v1.HealthCheckRequest)
    (*grpc_health_v1.HealthCheckResponse, error) {
    s.mu.RLock()
    defer s.mu.RUnlock()

    service := req.GetService()
    if service == "" {
        // Overall system health
    }
}
```



```

        return &grpc_health_v1.HealthCheckResponse{
            Status:
                grpc_health_v1.HealthCheckResponse_SERVING,
        }, nil
    }

    status, ok := s.services[service]
    if !ok {
        return &grpc_health_v1.HealthCheckResponse{
            Status:
                grpc_health_v1.HealthCheckResponse_SERVICE_UNKNOWN,
        }, nil
    }

    return &grpc_health_v1.HealthCheckResponse{Status:
        status}, nil
}

func (s *grpcHealthServer) Watch(req
    *grpc_health_v1.HealthCheckRequest, stream
    grpc_health_v1.Health_WatchServer) error {
    service := req.GetService()
    ch := make(chan
        grpc_health_v1.HealthCheckResponse_ServingStatus,
        10)

    s.mu.Lock()
    s.watchers[service] = append(s.watchers[service], ch)
    s.mu.Unlock()

    defer func() {
        s.mu.Lock()
        watchers := s.watchers[service]
        for i, w := range watchers {
            if w == ch {
                s.watchers[service] = append(watchers[:i],
                    watchers[i+1:]...)
                break
            }
        }
        s.mu.Unlock()
    }()

    for {
        select {
        case <-stream.Context().Done():
            return stream.Context().Err()
        case status := <-ch:

```

```

        if err :=
            stream.Send(&grpc_health_v1.HealthCheckResponse{Status:
                status}); err != nil {
                return err
            }
        }
    }
}

func (s *grpcHealthServer) SetServiceStatus(service string,
    status
    grpc_health_v1.HealthCheckResponse_ServingStatus) {
    s.mu.Lock()
    s.services[service] = status
    watchers := s.watchers[service]
    s.mu.Unlock()

    for _, ch := range watchers {
        select {
        case ch <- status:
        default:
            // Watcher channel full, skip notification
        }
    }
}

```

Test Command:

```

go test -race ./internal/health/... -run
    TestGRPCHealthCheck -v -tags=integration
go test -race ./internal/health/... -run
    TestGRPCHealthWatch -v -tags=integration

```

Acceptance Criteria: - gRPC health service is registered alongside HTTP server - Check RPC returns accurate service status - Watch RPC streams real-time health updates - Health events are published to the event bus - Integration test proves gateway can query gRPC health

CRIT-3: Fix Build Service Import Paths

Priority: P0 — CRITICAL

Estimated Effort: 8 person-hours

Current State: cmd/helix-build uses import paths that bypass the build orchestration layer in internal/build/. This means builds are not properly tracked, monitored, or cleaned up, and the orchestration layer's guarantees (retry, timeout, cancellation) are bypassed.

Root Cause Analysis:

```
// CURRENT CODE (cmd/helix-build/main.go – approximate)
import (
    "github.com/HelixDevelopment/helix_cluster/pkg/
        build"           // Direct package import
    // Missing: "github.com/HelixDevelopment/helix_cluster/
        internal/build" // Orchestrator
)

func main() {
    // BUG: Creates a PodmanBuilder directly, bypassing the
        orchestrator
    builder := build.NewPodmanBuilder(build.Config{...})
    result, err := builder.Build(context.Background(), spec)
    // ...
}
```

The `PodmanBuilder` is created directly instead of going through the `Orchestrator`, which provides:

- Build state machine (CREATING → BUILDING → COMPLETED/FAILED)
- Retry logic for transient failures
- Timeout enforcement
- Cancellation support
- Build status tracking
- Worker pool management
- Artifact storage

Implementation:

1. Fix `cmd/helix-build/main.go` to import through `internal/build`
2. Remove `SimulatedBuilder` from the production path (it should only be used in tests)
3. Add integration test verifying the orchestrator is used

```
// FIXED CODE (cmd/helix-build/main.go)
import (
    "github.com/HelixDevelopment/helix_cluster/internal/
        build"
)

func main() {
    cfg := build.LoadConfig()

    // Create orchestrator (not a raw builder)
    orchestrator :=
        build.NewOrchestrator(build.OrchestratorConfig{
            MaxWorkers:      cfg.MaxWorkers,
            BuildTimeout:    cfg.BuildTimeout,
            RetryCount:      cfg.RetryCount,
            ArtifactDir:    cfg.ArtifactDir,
        })
}
```

```

// Register builders through the orchestrator
orchestrator.RegisterBuilder("podman",
    build.NewPodmanBuilder(cfg.Podman))
orchestrator.RegisterBuilder("go",
    build.NewGoBuilder(cfg.Go))
// DO NOT register SimulatedBuilder in production

// Create gRPC server using the orchestrator
server := build.NewServer(orchestrator, cfg)

// Start server
if err := server.Start(); err != nil {
    log.Fatalf("failed to start build server: %v", err)
}
}

```

Test Command:

```

go test -race ./internal/build/... -run
    TestOrchestratorPath -v
go test -race ./cmd/helix-build/... -run
    TestNoSimulatedBuilder -v

```

Acceptance Criteria: - Build service imports through `internal/build`
 orchestrator - `SimulatedBuilder` is not registered in production path -
 Integration test proves orchestrator state machine is followed - Build
 status is tracked through the orchestrator

Phase 7: Documentation Continuous

Duration: Ongoing

Owner: Autonomous

Prerequisite: All phases (documentation updates follow code changes)

Gate: `docs_chain` verify passes

Estimated Effort: ~120 person-hours (over the life of the project)

Objective

Implement continuous documentation sync per CLAUDE-3. Every code change must trigger a documentation update. The `docs_chain` engine is the mechanical enforcer.

PH7-001: docs_chain engine integration

Priority: P1

Estimated Effort: 24 person-hours

1. Register all materials in `.docs_chain/contexts/`
2. Add `docs_chain verify` to pre-build gate
3. Ensure `docs_chain` runs with every change
4. No escape hatch — no `--skip-docs-chain`

PH7-002: Export format pipeline (MD→PDF→HTML→DOCX)

Priority: P2

Estimated Effort: 40 person-hours

Implement the multi-format export pipeline:

```
scripts/docs/export.sh --input docs/ --output exports/ --  
formats pdf,html,docx
```

Tools: - MD → PDF: `pandoc` with custom LaTeX template - MD → HTML: `mkdocs` with material theme - MD → DOCX: `pandoc` with custom Word template

PH7-003: Diagram generation automation

Priority: P2

Estimated Effort: 16 person-hours

Automate Mermaid diagram generation from code:

```
scripts/docs/generate-diagrams.sh --source ./ --output docs/  
diagrams/
```

Diagrams to auto-generate: 1. Service dependency graph (from `internal/ imports`) 2. Package dependency graph (from `pkg/ imports`) 3. Database ER diagram (from migrations) 4. API surface diagram (from proto definitions) 5. Configuration flow diagram (from config structs)

Phase 8: Bucket B Infrastructure

Duration: 12–16 weeks

Owner: OWNER-PROVISIONED (requires hardware, cloud accounts, tokens)

Prerequisite: Phases 1–6 complete

Gate: All 152 Bucket B items resolved
Estimated Effort: ~1,200 person-hours

Objective

Resolve the 152 infrastructure/hardware/cloud-blocked items from the HXC registry. These items require owner-provisioned resources that cannot be obtained autonomously.

Resource Requirements

Resource	Purpose	Items Blocked
Kubernetes cluster (3+ nodes)	E2E testing, service mesh	~30
SBC hardware (RPi, RK3588)	Edge device testing	~15
GPU hardware (NVIDIA, AMD, Intel)	GPU backend testing	~25
Cloud accounts (AWS, GCP, Azure)	Cloud deployment testing	~20
macOS CI runner	Cross-platform testing	~10
SonarQube token	Code quality analysis	~5
Snyk token	Dependency scanning	~5
NVIDIA MIG hardware	MIG partitioning tests	~10
PS4/PS5 hardware	Console integration	~5
Multi-host network (3+ nodes)	Network chaos testing	~10

Sub-Phases

Phase 8A: Kubernetes Deployment (4-6 weeks)

Deploy all 14 microservices to Kubernetes with proper service mesh, Helm charts, and E2E testing. This sub-phase requires a Kubernetes cluster with at least 3 nodes and sufficient GPU resources.

Infrastructure Requirements: - Kubernetes 1.28+ cluster with 3+ nodes - Helm 3.12+ for chart management - Istio 1.20+ or Linkerd 2.14+ for service mesh - cert-manager for TLS certificate management - NVIDIA

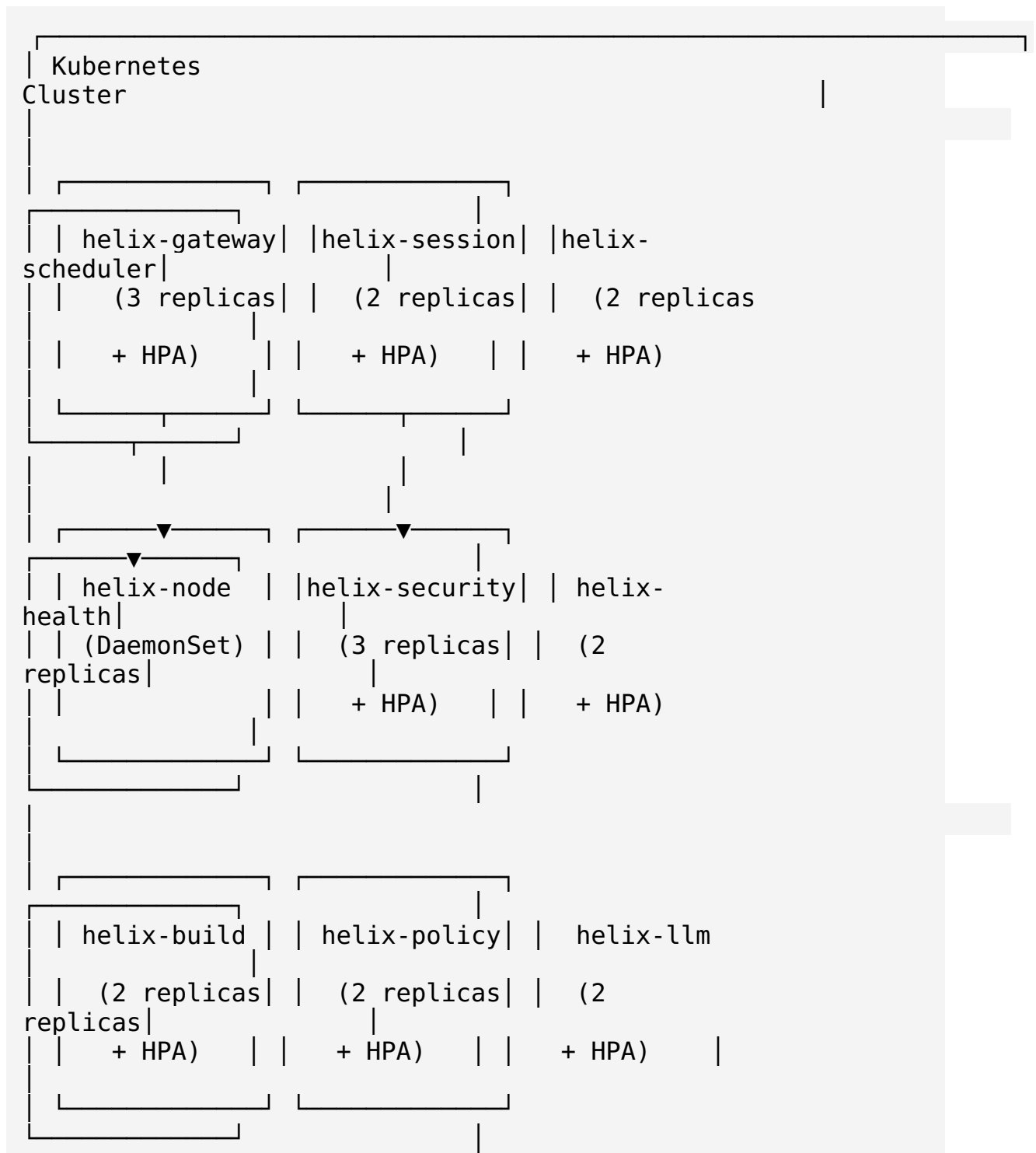
GPU Operator (for GPU nodes) - Prometheus + Grafana for monitoring - ArgoCD or Flux for GitOps deployment

Work Items:

Item	Description	Effort
PH8A-001	Create Helm chart for helix-gateway	8h
PH8A-002	Create Helm chart for helix-session	8h
PH8A-003	Create Helm chart for helix-scheduler	8h
PH8A-004	Create Helm chart for helix-node	6h
PH8A-005	Create Helm chart for helix-security	6h
PH8A-006	Create Helm chart for helix-health	4h
PH8A-007	Create Helm chart for helix-build	6h
PH8A-008	Create Helm chart for helix-policy	4h
PH8A-009	Create Helm chart for helix-llm	4h
PH8A-010	Create Helm chart for helix-advisory	4h
PH8A-011	Create Helm chart for helix-agent	6h
PH8A-012	Create Helm chart for e2ee-proxy	4h
PH8A-013	Create Helm chart for htmux	4h
PH8A-014	Create Helm chart for gpu-pool-manager	4h
PH8A-015	Create umbrella Helm chart for full deployment	8h
PH8A-016	Configure Istio service mesh with mTLS	8h
PH8A-017	Configure cert-manager with SPIFFE issuer	6h
PH8A-018	Configure NVIDIA GPU Operator	4h
PH8A-019	Deploy Prometheus + Grafana monitoring stack	6h
PH8A-020	Create K8s-specific configuration for all services	8h
PH8A-021	Implement rolling deployment strategy	6h
PH8A-022	Implement canary deployment for scheduler	8h
PH8A-023	Create K8s health probes for all services	6h

Item	Description	Effort
PH8A-024	Configure resource limits and requests	4h
PH8A-025	E2E testing on K8s (all 14 services)	16h
PH8A-026	Rolling deployment testing	8h
PH8A-027	Disaster recovery testing on K8s	8h

Kubernetes Deployment Architecture:




```
| Istio Service Mesh (mTLS, traffic management,  
observability)| |
```

```
| NVIDIA GPU Operator + GPU Resource  
Manager| |
```

Phase 8B: GPU Hardware Testing (4-6 weeks)

Test all GPU backends on real hardware to verify the GPU management stack works across all 4 vendor backends. This sub-phase requires physical access to GPU hardware or cloud GPU instances.

Hardware Requirements:

Vendor	Hardware	Minimum	Purpose
NVIDIA	A100/H100 GPUs	2× GPUs	CUDA, NVML, MIG profiling
NVIDIA	RTX 4090 GPUs	1× GPU	Consumer GPU testing
AMD	MI250X/MI300X GPUs	1× GPU	ROCm backend
Apple	M3 Pro/Max GPU	1× Mac	Metal backend
Intel	Arc A770 / Data Center GPU	1× GPU	oneAPI/SYCL backend

Work Items:

Item	Description	Effort	Hardware
PH8B-001		8h	NVIDIA GPU

Item	Description	Effort	Hardware
	NVIDIA CUDA kernel execution test		
PH8B-002	NVIDIA NVML monitoring integration test	6h	NVIDIA GPU
PH8B-003	NVIDIA MIG partitioning test (A100/H100)	12h	NVIDIA A100/H100
PH8B-004	NVIDIA GPU allocation/deallocation test	6h	NVIDIA GPU
PH8B-005	NVIDIA GPU attestation (PoVW) test	8h	NVIDIA GPU
PH8B-006	AMD ROCm backend integration test	12h	AMD GPU
PH8B-007	AMD GPU monitoring integration test	6h	AMD GPU
PH8B-008	Apple Metal backend integration test	8h	Apple Mac
PH8B-009	Apple GPU allocation test	4h	Apple Mac
PH8B-010	Intel oneAPI/SYCL backend test	12h	Intel GPU
PH8B-011	Multi-GPU gang scheduling test	16h	2+ NVIDIA GPUs
PH8B-012	GPU failover test (GPU fault injection)	8h	NVIDIA GPU
PH8B-013	GPU thermal throttling test	6h	NVIDIA GPU
PH8B-014		8h	NVIDIA GPU

Item	Description	Effort	Hardware
	GPU memory leak soak test (24h)		
PH8B-015	Cross-vendor GPU scheduling test	8h	Multiple GPUs

Phase 8C: Cross-Platform Testing (2-4 weeks)

Test all platform-specific implementations on real hardware. This requires macOS CI runners, Windows/WSL2 machines, ARM64 hardware, and edge devices.

Work Items:

Item	Description	Effort	Platform
PH8C-001	macOS CI runner setup (GitHub Actions)	8h	macOS
PH8C-002	macOS full test suite run	4h	macOS
PH8C-003	macOS WireGuard mesh test	8h	macOS
PH8C-004	macOS session backend test	4h	macOS
PH8C-005	macOS GPU detection test (IOKit)	6h	macOS
PH8C-006	Windows/WSL2 test setup	8h	Windows
PH8C-007	Windows session backend (ConPTY) test	8h	Windows
PH8C-008	ARM64 cross-compilation test	4h	ARM64
PH8C-009	ARM64 runtime test on RPi4	6h	ARM64
PH8C-010	RK3588 edge device test	8h	ARM64
PH8C-011	PS4/PS5 console detection test	8h	Console
PH8C-012	Cross-platform resource monitoring test	6h	All

Phase 8D: Cloud Deployment (2-4 weeks)

Test cloud deployment on AWS, GCP, and Azure with multi-cloud federation.

Cloud Account Requirements: - AWS account with EC2 GPU instances (P4d/P5 for NVIDIA A100/H100) - GCP account with Compute Engine GPU instances (A100) - Azure account with NC A100 v4 series

Work Items:

Item	Description	Effort	Cloud
PH8D-001	AWS deployment with Terraform	12h	AWS
PH8D-002	AWS GPU instance testing	8h	AWS
PH8D-003	GCP deployment with Terraform	12h	GCP
PH8D-004	GCP GPU instance testing	8h	GCP
PH8D-005	Azure deployment with Terraform	12h	Azure
PH8D-006	Azure GPU instance testing	8h	Azure
PH8D-007	Multi-cloud federation test	16h	All
PH8D-008	Cloud cost optimization testing	4h	All
PH8D-009	Cloud disaster recovery test	8h	AWS
PH8D-010	Cloud auto-scaling test	8h	AWS

Phase 9: Bucket C Governance

Duration: 2–4 weeks

Owner: OWNER-DECISION (requires governance rulings)

Prerequisite: Phases 1–7 complete

Gate: All 10 Bucket C items resolved

Estimated Effort: ~40 person-hours

Objective

Resolve the 10 governance/owner-decision items from the HXC registry. These items require explicit rulings from the project owner.

PH9-001: CI re-enable decision (HXC-105/HXC-1262)

Priority: P0 — GOVERNANCE

Estimated Effort: 2 person-hours (implementation) + owner decision time

Current State: All GitHub Actions workflows are stored under `.github/workflows/disabled/`. The project constitution mandates a no-CI rule.

Decision Required: Should CI be re-enabled? If yes, which workflows first?

Recommendation: Re-enable in the following order: 1. `go-build.yml` — Build verification 2. `go-test.yml` — Test execution 3. `format.yml` — Code formatting 4. `lint.yml` — Linting 5. `race.yml` — Race detection 6. `security-scan.yml` — Security scanning 7. `dst-sim.yml` — Deterministic simulation 8. `release.yml` — Release pipeline 9. `docker-build.yml` — Container image builds 10. `docs.yml` — Documentation verification 11. `vm_integration.yml` — VM integration tests 12. `cc-build.yml` — C/C++ build

Impact if re-enabled: PRR items 8/9/10/16/60 become closeable, raising PRR by ~7.5%.

PH9-002: PRR $\geq$ 95% closure

Priority: P0 — GOVERNANCE

Estimated Effort: 4 person-hours

After all previous phases are complete, assess the PRR score. If it's still below 95%, identify the remaining gaps and create a targeted plan to close them.

Current PRR gaps:

Item	Description	Blocker
7	Coverage gate not enforced	pkg/covgate exists but unwired
8	CI build/test not active	CI disabled (HXC-105)
9	Release pipeline not active	CI disabled (HXC-105)
10	Coverage gate not enforced	Same as #7
16	Dependency scanning not continuous	CI disabled
21	HelixQA challenges not executed	Challenge bank not populated (Phase 3)
33	mTLS e2e incomplete	Requires real SPIFFE CA (Phase 6)
34		CI disabled

Item	Description	Blocker
	Dep-scan continuous gate	
60	Integration test gate not enforced	CI disabled

PH9-003: Release authorization

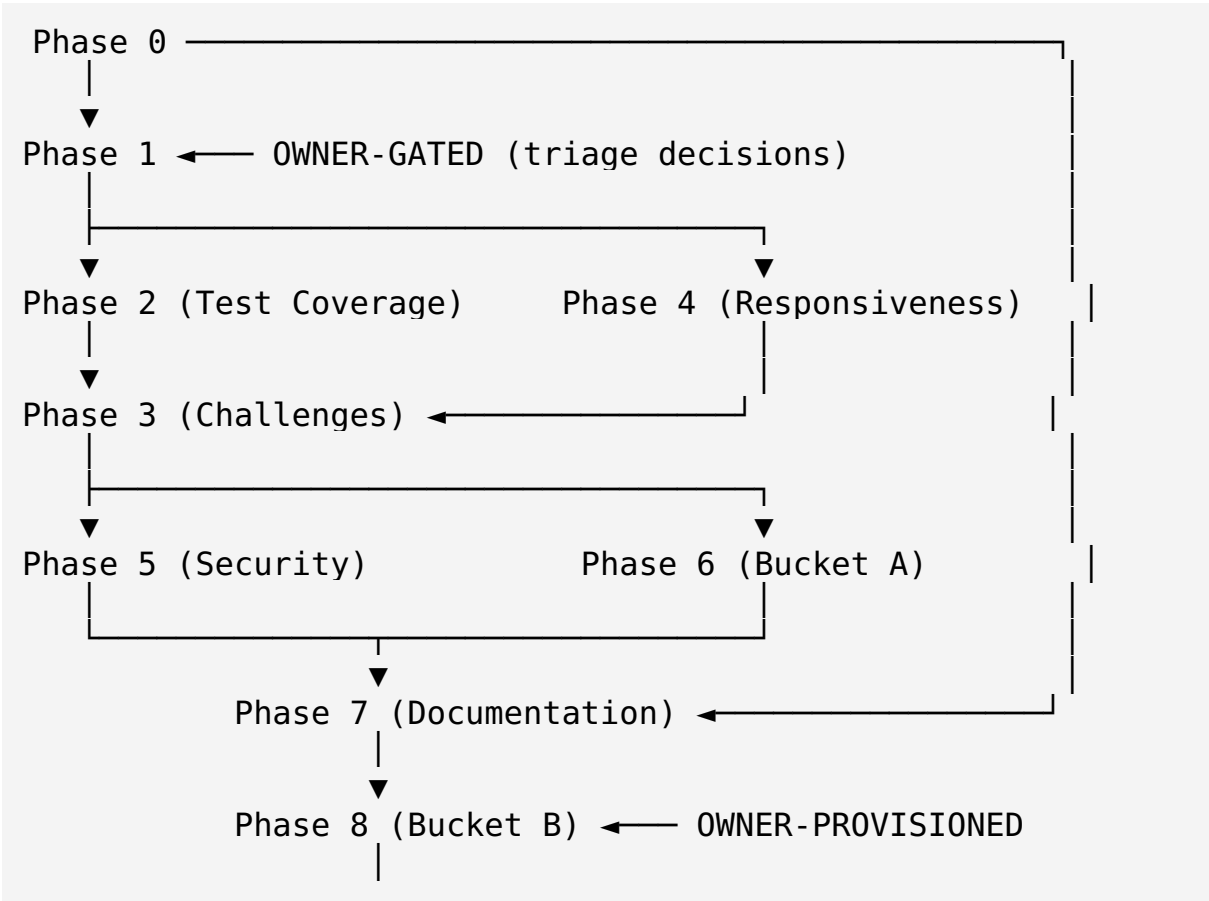
Priority: P0 — GOVERNANCE
Estimated Effort: 2 person-hours

Once PRR $\geq$ 95% and all CRITICAL items are resolved, the owner must authorize the first release.

Release checklist: - ☐ All Phase 0-8 items complete - ☐ PRR $\geq$ 95% - ☐ All challenges passing - ☐ All security scans clean - ☐ Documentation synced - ☐ CI pipeline active and green - ☐ Performance baselines established

Cross-Phase Dependencies

Dependency Graph



Parallel Execution Opportunities

The following phases can run in parallel after Phase 0:

Parallel Track	Phases	Rationale
Track A	Phase 1 → Phase 2 → Phase 3	Main critical path
Track B	Phase 4 (Responsiveness)	Independent of wiring
Track C	Phase 5 (Security scanning)	Independent of wiring
Track D	Phase 7 (Documentation)	Can start immediately

Risk Register

ID	Risk	Probability	Impact	Mitigation
R1	Owner does not respond to triage decisions	Medium	Blocks Phase 1	Default to DOCUMENT-AS-LIBRARY after 2-week timeout
R2	New concurrency hazards discovered during wiring	High	Delays Phase 1	Fix immediately in Phase 0 gate style
R3	Wired packages have more stubs than expected	Medium	Requires anti-bluff fixes	Budget for stub replacement in each wiring item
R4	Test infrastructure (etcd, PostgreSQL, Redis) unavailable	Low	Blocks integration tests	Use testcontainers-go for ephemeral containers
R5		High	Blocks Phase 8B	Use mock GPU backends for

ID	Risk	Probability	Impact	Mitigation
	GPU hardware unavailable for testing			unit/integration; defer real hardware tests
R6	CI remains disabled after Phase 9	Medium	PRR stays below 95%	Local enforcement via make gate-check
R7	Documentation drift during rapid development	High	CLAUDE-3 violation	docs_chain verify as pre-commit hook
R8	Challenge framework incompatibility	Low	Blocks Phase 3	Test with simple challenges first
R9	Owner does not provision resources for Phase 8	Medium	Phase 8 stalls indefinitely	Proceed with mock/simulation-based testing
R10	Regression in existing tests	Medium	All phases delayed	Phase 0 gate enforced before every merge

Resource Estimation

Person-Hours by Phase

Phase	Autonomous Hours	Owner Hours	Total
Phase 0	160	0	160
Phase 1	400	80	480
Phase 2	640	0	640
Phase 3	320	0	320
Phase 4	80	0	80
Phase 5	60	0	60

Phase	Autonomous Hours	Owner Hours	Total
Phase 6	960	0	960
Phase 7	120	0	120
Phase 8	800	400	1,200
Phase 9	0	40	40
Total	3,540	520	4,060

Timeline

Optimistic (all owner decisions immediate, all resources available)

- Phase 0: Weeks 1-3
- Phase 1: Weeks 4-9
- Phase 2: Weeks 10-17
- Phase 3: Weeks 18-23
- Phase 4: Weeks 10-12 (parallel with Phase 2)
- Phase 5: Weeks 10-12 (parallel with Phase 2)
- Phase 6: Weeks 24-35
- Phase 7: Continuous
- Phase 8: Weeks 24-39 (parallel with Phase 6)
- Phase 9: Weeks 40-43

Total: ~43 weeks (10 months)

Realistic (owner decisions take 1-2 weeks, some resources delayed)

- Phase 0: Weeks 1-3
- Phase 1: Weeks 4-10 (owner gate delays)
- Phase 2: Weeks 11-18
- Phase 3: Weeks 19-24
- Phase 4: Weeks 11-13 (parallel)
- Phase 5: Weeks 11-13 (parallel)
- Phase 6: Weeks 25-36
- Phase 7: Continuous
- Phase 8: Weeks 25-42 (resource delays)
- Phase 9: Weeks 43-46

Total: ~46 weeks (11 months)

Pessimistic (owner decisions take 4+ weeks, resources severely delayed)

Total: ~68 weeks (16 months)

Appendix A: Item Registry Cross-Reference

Plan Item	HXC Ticket	Gap Audit ID	Constitution Rule
PH0-001	HXC-1639	Gap-6 (Schema Drift)	§11.4.106
PH0-002	—	F1	§7.1
PH0-003	—	F2	§7.1
PH0-004	—	F3	§7.1
PH0-005	—	F4	§7.1
PH0-006	—	F5	§7.1
PH0-007	—	F6	§7.1
PH0-008	—	F7	§7.1
PH1-001	—	Gap-1 (Orphaned Code)	CLAUDE-1
PH1-002	—	Gap-1 (Orphaned Code)	CLAUDE-1
PH2-001	—	Gap-4 (Test Depth)	§1.1, §7.1
PH2-002	—	Gap-4 (Fuzz Tests)	§11.4
PH3-001	—	—	CONST-050
PH4-001	—	F8	§7.1
PH4-002	—	F9	§7.1
PH4-003	—	F10	§7.1
PH5-001	—	Gap-5 (Security Scanning)	§11.4
	HXC-1213	Gap-3 (CRIT-1)	CLAUDE-1

Plan Item	HXC Ticket	Gap Audit ID	Constitution Rule
PH6-CRIT1			
PH6-CRIT2	—	Gap-3 (CRIT-2)	CLAUDE-1
PH6-CRIT3	—	Gap-3 (CRIT-3)	CLAUDE-1
PH9-001	HXC-105, HXC-1262	Gap-6 (CI)	Constitution

Appendix B: Constitution Compliance Map

Constitution Rule	Current Status	Phase Addressed	Target Status
PCS-1 (Cross-Platform)	PARTIAL	Phase 8C	COMPLIANT
PCS-2 (GPU Backend)	NON-COMPLIANT	Phase 8B	COMPLIANT
PCS-3 (No Hardcoded Secrets)	COMPLIANT	—	COMPLIANT
PCS-4 (Container Mandate)	PARTIAL	Phase 8A	COMPLIANT
PCS-6.1 (No Orphaned Code)	NON-COMPLIANT	Phase 1	COMPLIANT
PCS-6.2 (Coverage Gate)	NON-COMPLIANT	Phase 2	COMPLIANT
CLAUDE-1 (End-User Usability)	NON-COMPLIANT	Phases 1, 6	COMPLIANT
CLAUDE-2 (Cross-Platform)	PARTIAL	Phase 8C	COMPLIANT
	PARTIAL	Phase 7	COMPLIANT

Constitution Rule	Current Status	Phase Addressed	Target Status
CLAUDE-3 (Documentation Sync)			
§1.1 (Mutation Tests)	NON-COMPLIANT	Phase 2	COMPLIANT
§7.1 (Quality Guarantee)	NON-COMPLIANT	Phase 9	COMPLIANT
§11.4 (Anti-Bluff)	NON-COMPLIANT	Phases 2, 3	COMPLIANT
CONST-035 (Anti-Bluff Covenant)	NON-COMPLIANT	Phase 3	COMPLIANT
CONST-050 (100% Test Coverage)	NON-COMPLIANT	Phase 2	COMPLIANT

Appendix C: Glossary

Term	Definition
Anti-bluff	A test or challenge that verifies the feature actually works, not just that code executes without panic
Bucket A	HXC registry items that are autonomously actionable
Bucket B	HXC registry items blocked by infrastructure/hardware/cloud requirements
Bucket C	HXC registry items requiring governance/owner decisions
CRIT	CRITICAL severity — must be fixed before any release
DST	Deterministic Simulation Testing — FoundationDB-style testing
F1-F10	Concurrency hazard identifiers (F1=Major through F10=Minor)

Term	Definition
HXC	Helix Cluster ticket system
KAT	Known Answer Test — cryptographic test vectors
OCC	Optimistic Concurrency Control
Omega model	Shared-state scheduling with multiple independent schedulers
Orphaned package	A pkg/ package with zero importers from any binary
PASS-bluff	A test that passes on non-functional code — §7.1 violation
PRR	Production Readiness Review — 80-item checklist
Sink-side evidence	Evidence captured at the output/result side, proving the feature actually produced correct output
SWIM	Scalable Weakly-consistent Infection-style Process Group Membership
sync.Once	Go synchronization primitive ensuring a function is called exactly once
WaitGroup	Go synchronization primitive for waiting on a group of goroutines
Wire-in	Connect an orphaned package to a binary so it becomes reachable

End of Phased Remediation Plan

Document Statistics: - Total phases: 9 - Total work items: ~354 - Total estimated effort: ~4,060 person-hours - Owner-gated items: ~80 person-hours - Owner-provisioned items: ~400 person-hours - Owner-decision items: ~40 person-hours - Autonomous items: ~3,540 person-hours - Constitution rules addressed: 14 - Concurrency hazards resolved: 10 - CRITICAL disconnections resolved: 3 - Orphaned packages triaged: 178